



Name	Student ID
Ian Lim	TP073235
Chan Kar Jun	TP074808
Ang Kuan Hern	TP074844
Dya Singh Gill A/L Rajinder Singh Gill	TP074871

Intake Code: UCDF2304ICT(SE)

Subject: Programming with Python

Title: Group Assignment - Documentation

Date Assigned: 4th April 2024

Date Completed: 1st June 2024

Table of Contents

1.0 Workload Matrix.....	4
2.0 Introduction and Assumptions	5
3.0 Design of Program (Flowchart and Pseudocode)	6
3.1 Manager	6
3.1.1 Main Menu.....	6
3.1.2 Login	9
3.1.3 Register and Update.....	12
3.1.4 Delete Staff	23
3.1.5 Set and update car renting rate.....	27
3.1.6 View monthly revenue report	31
3.1.7 Update profile	33
3.2 Customer Service Staff 1	37
3.2.1 Main Menu.....	37
3.2.2 Menu 1	38
3.2.3 Register Customer.....	40
3.2.4 Update Customer Details	41
3.2.5 View List of Registered Customers	44
3.2.6 Updated Profile	45
3.2.7 Delete Customers who have no rental transaction recorded.....	48
3.3 Customer Service Staff 2	50
3.3.1 Menu 2	50
3.3.2 Check Availability	54
3.3.3 Accept and Record Rental Details	56
3.3.4 Generate Bills.....	58

3.3.5 Accept Payment from Customer and Generate Receipt	60
3.3.6 Update “Reserved” when Customer Books Car	62
3.3.7 Update “Rented” when Customer Rents Car	64
3.3.8 Update “Available” when Customer Return Car	66
3.3.9 View List Rental Transactions.....	68
3.3.10 Delete the Cancelled Car Rental Transaction	70
3.4 Car Service Staff.....	72
3.4.1 Main Menu.....	72
3.4.2 Register new cars	79
3.4.3 Update Car Details	81
3.4.4 View Lists of Cars	89
3.4.5 Delete Car where availability is “Disposed”	90
3.4.6 Update Profile	92
4.0 Explanation of Programming Concepts	95
5.0 Additional Features Source Code with Explanation.....	100
6.0 Sample Input/Output with Explanation	104
7.0 Conclusion	109
8.0 References.....	110

1.0 Workload Matrix

Role / Task	Name
Manager	Ang Kuan Hern
Customer Service Staff 1	Dya Singh
Customer Service Staff 2 Documentation	Ian Lim
Car Service Staff	Chan Kar Jun

2.0 Introduction and Assumptions

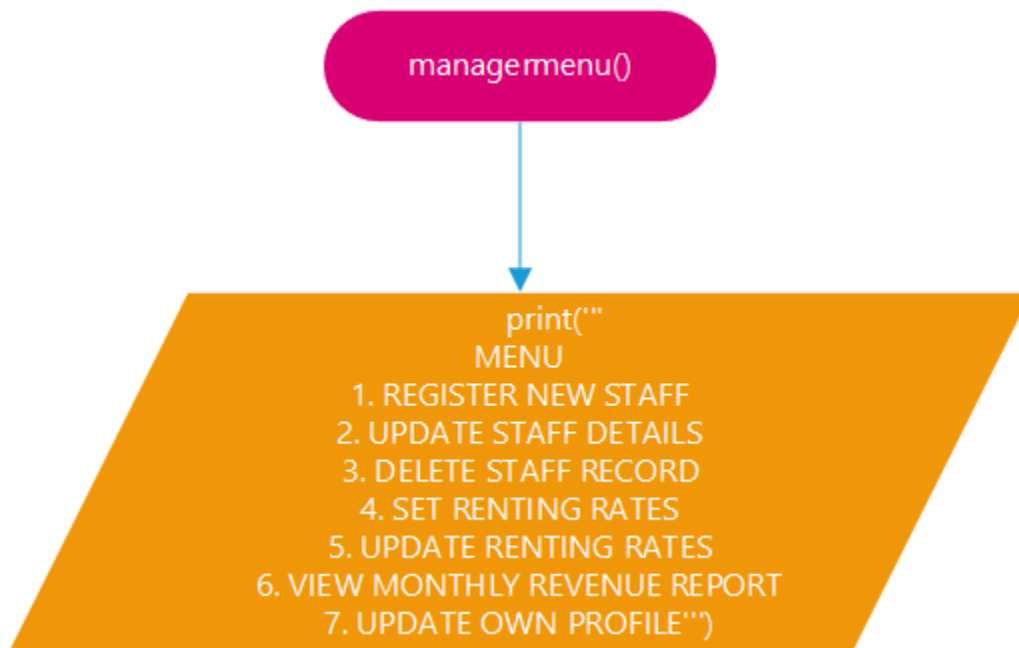
In today's fast-paced world, businesses are continuously looking for methods to improve their operations and consumer happiness. One such business is car rental, where efficiency and dependability are critical. The manager of a car rental firm on Langkawi Island has seen the need to modernise their business operations in order to meet increased demand and competition. To solve this, the management has charged us with creating a Python programme that simulates a full car rental system according to the company's needs.

To guarantee that the car rental system runs well, certain assumptions have been established. To begin, user authentication is expected to be secure, with users, including managers and staff, authenticating themselves using a username and password, and the system implementing access restriction based on user responsibilities. Second, data integrity is critical, and the system must ensure accuracy and consistency of user and car records, such as staff data, customer profiles, and rental transactions. Error handling systems are considered to be strong, with features like restricting staff login attempts and ensuring data integrity across the system. Customers will be required to input personal information upon registration, and the system will automatically generate unique customer IDs. Similarly, car registration requires entering critical information such as the registration number, manufacturer, and maintenance time, with the system automatically assigning availability statuses depending on rental activity. Rental transactions will be precisely documented, including rental dates, customer IDs, and billing information, and the system will calculate rental costs based on established rates and rental conditions. Finally, reporting functions will include monthly income reports for managers and access to lists of registered customers, available cars, and rental transactions as needed. The Python programme constructed using these assumptions and features attempts to streamline the company's car rental procedure, boosting efficiency and customer service.

3.0 Design of Program (Flowchart and Pseudocode)

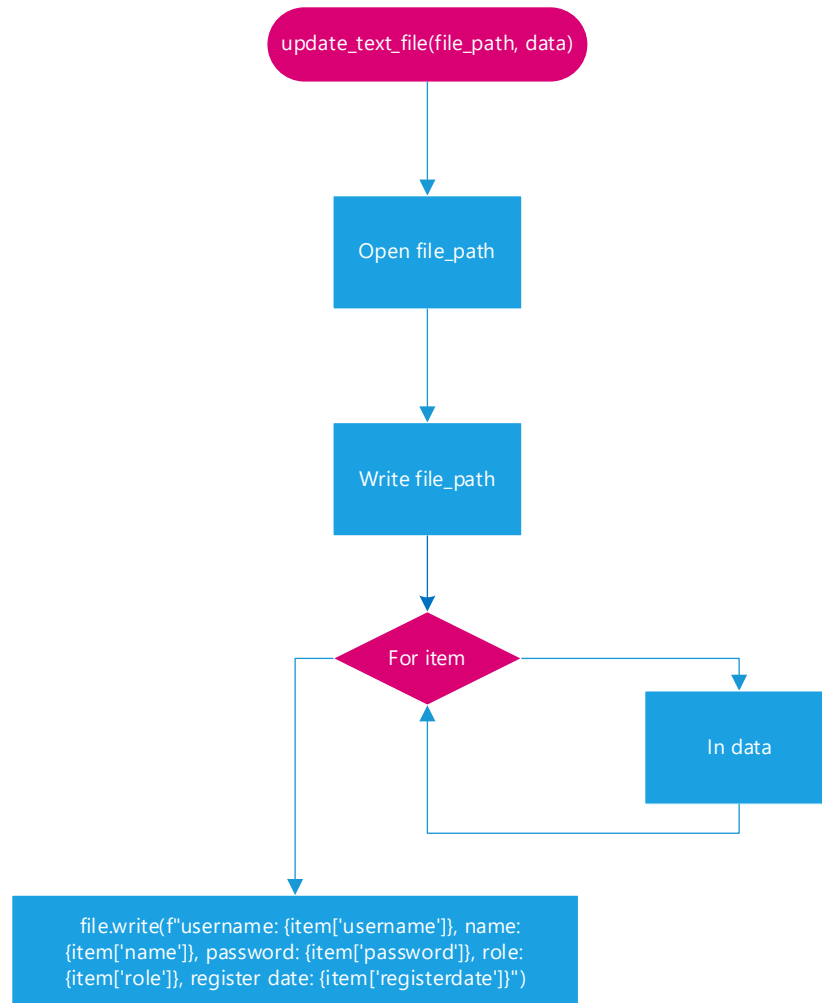
3.1 Manager

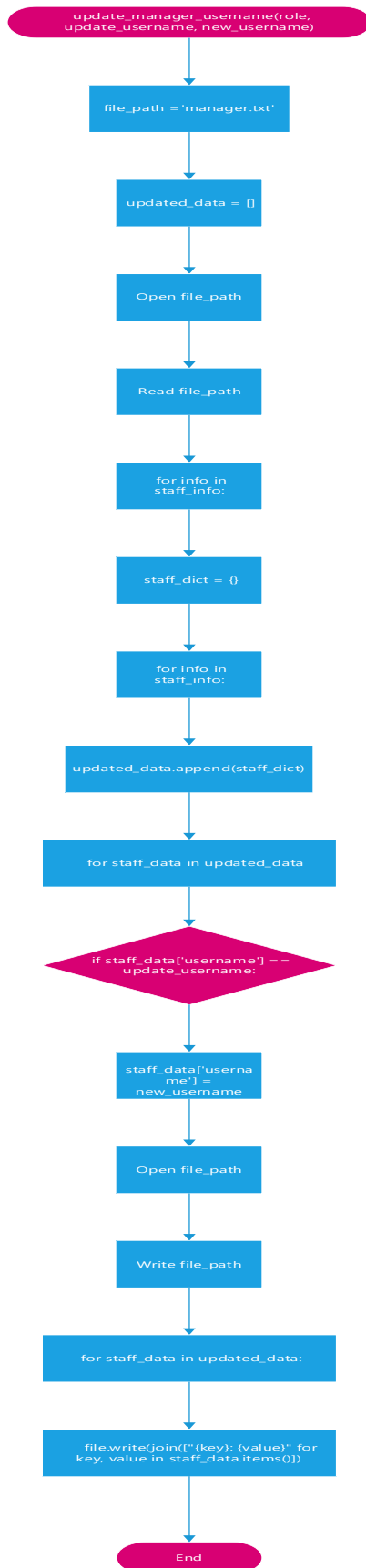
3.1.1 Main Menu



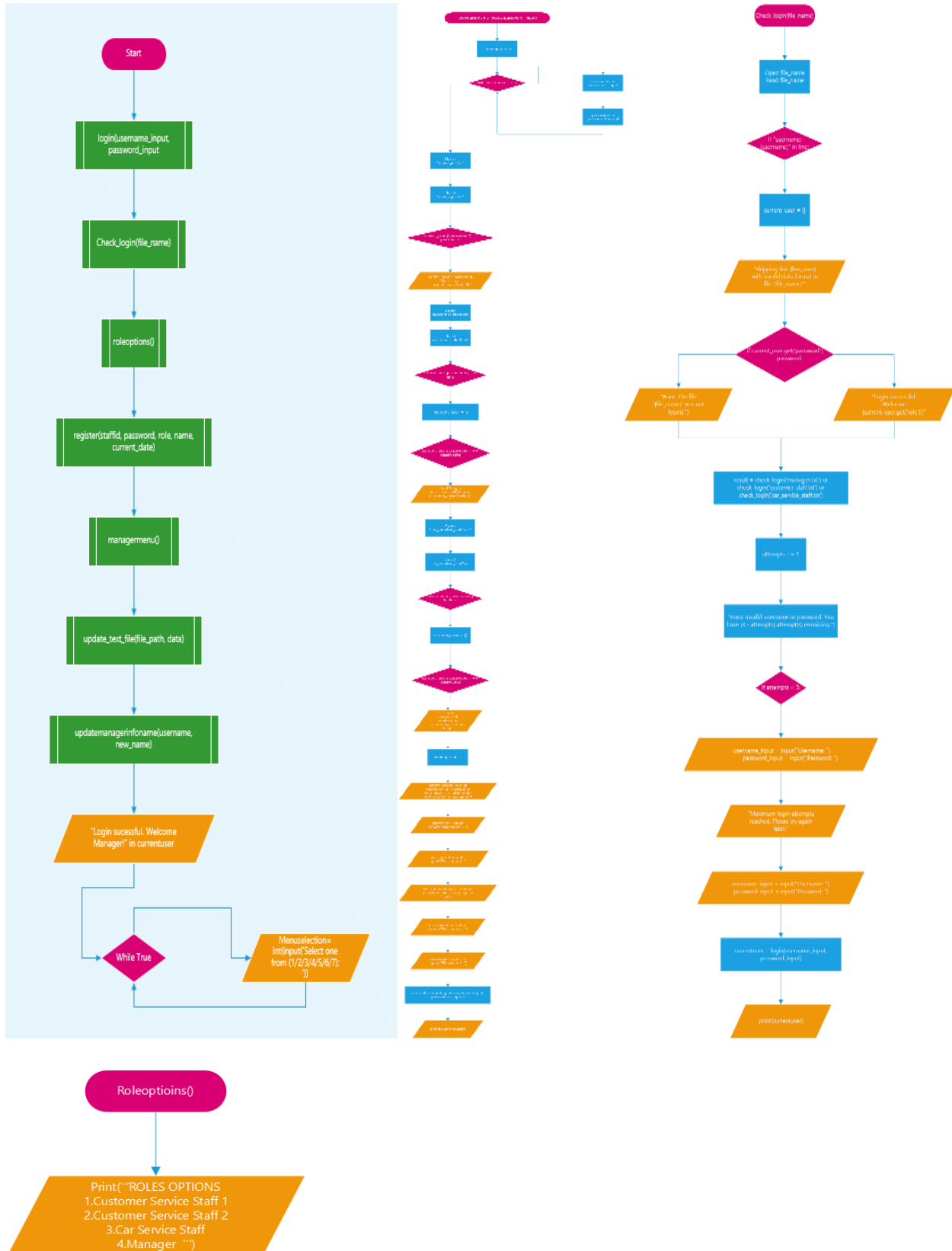
Define managemenu()

```
Print ("
MENU
1. REGISTER NEW STAFF
2. UPDATE STAFF DETAILS
3. DELETE STAFF RECORD
4. SET RENTING RATES
5. UPDATE RENTING RATES
6. VIEW MONTHLY REVENUE REPORT
7. UPDATE OWN PROFILE")
```





3.1.2 Login



Start

Define Login Function

Initialize login function with parameters username_input, password_input

Declare global variable username

Initialize attempts counter to 0

Start a while loop with the condition attempts < 2

Assign username_input to the global variable username

Assign password_input to a local variable password

Increment attempts by 1 after each iteration

End of while loop

Open "manager.txt"

Read from "manager.txt"

If "username: {username}" in "manager.txt":

Current_user = {}

If current_user "password" == password:

Return "Login successful. Welcome {current_user}['role']!"

Open "customer_staff.txt"

Read from "customer_staff.txt"

IF "username: {username}" in "customer_staff_1"

Current_user = {}

If current_user "password" == password:

Return "Login successful. Welcome {current_user}['role']!"

Open "car_service_staff.txt"

Read from "car_service_staff.txt"

IF "username: {username}" in "car_service_staff.txt"

Current_user = {}

If current_user "password" == password:

Return "Login successful. Welcome {current_user}['role']!"

Attempt += 1

Print ("Error: Invalid username or password. You have")

username_input = input("Username: ")

password_input = input("Password: ")

return "Maximum login attempts reached. Please try again later."

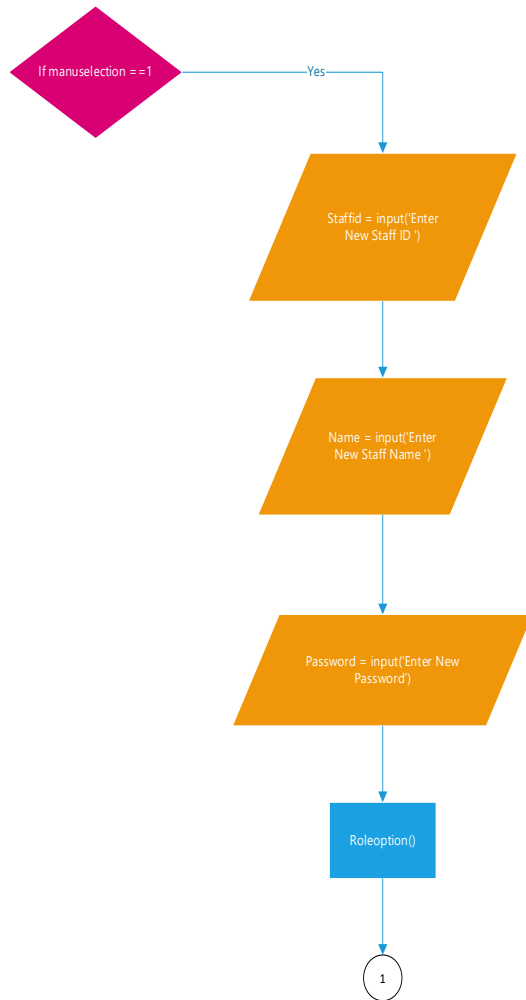
username_input = input("Username: ")

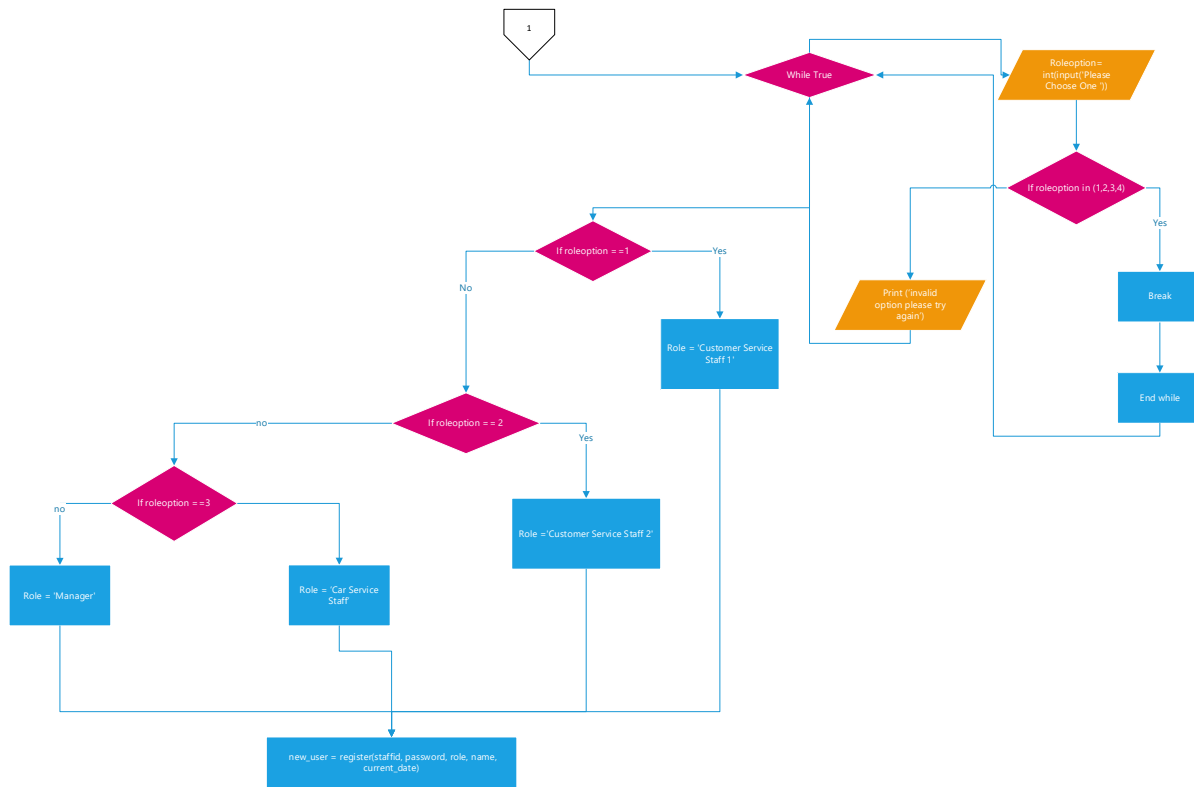
password_input = input("Password: ")

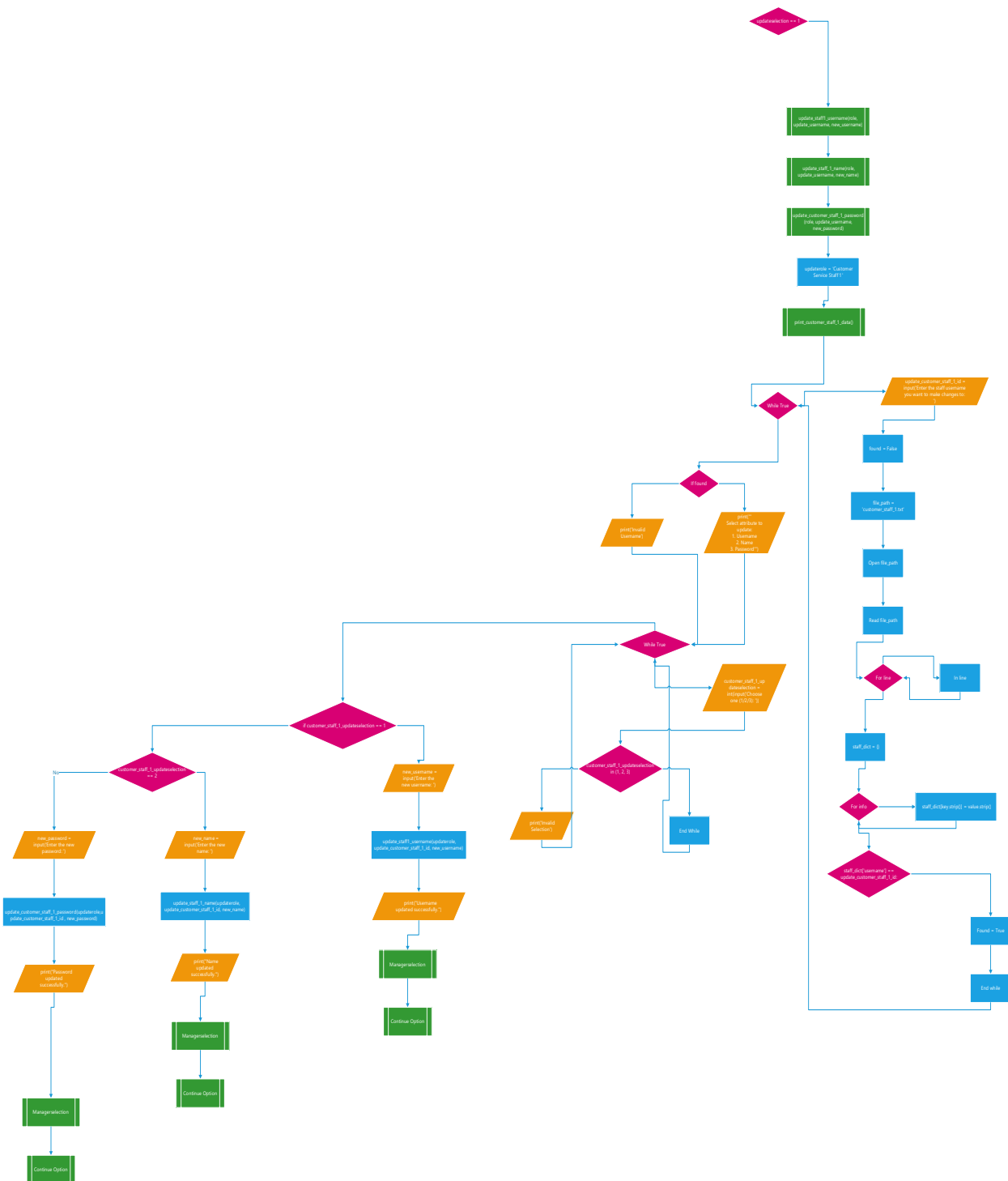
Call the login function

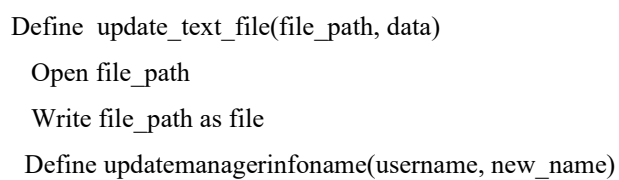
```
Currentuser = login(username_input, password_input)
Print (currentuser)
Define roleoptions
    Print(" ROLES OPTIONS
    1.Customer Service Staff
    2.Car Service Staff
    3.Manager ")
    If role == "Customer Service Staff 1"
Open 'customer_staff.txt'
Append 'customer_staff.txt' as file
    Elif role == "Car Service Staff"
Open 'Car Service Staff'
Append 'Car Service Staff' as file
    ELIF role == 'Manager'
Open 'Manager'
Append 'Manager' as file
```

3.1.3 Register and Update









```
Updated_manager_list=[]
Open 'manager.txt'
Read 'manager.txt' as file
Open 'manager.txt'
Write 'manager.txt' as file
If "Login successful. Welcome Manager!" in currentuser
```

```
Managemenu()
If Menuselection == 1
    Staffid = input 'Enter New Staff ID: '
    Name = input 'Enter New Staff Name: '
    Password = input 'Enter New Password: '
    Roleoption()
    While True:
        Roleoption = int(input('Please Choose One: '))
        If roleoption in (1,2,3,4)
            Break
        Else:
            Print 'invalid option please try again'
    If roleoption == 1
        Role = 'Customer Service Staff 1'
    Elif roleoption == 2
        Role = 'Customer Service Staff 2'
    Elif roleoption == 3
        Role = 'Car Service Staff'
    Elif roleoption == 4
        Role = 'Manager'
    Elif menuselection == 2
        Print (""
Select role:
1. Customer Service Staff
2. Car Service Staff
3. Manager")
While True
    Updateselection = int(input('Choose one (1/2/3): '))
    If updateselection in (1,2,3):
        Break
```



```
Else:
    Print ('Invalid Selection')
If updateselection == 1
    Define update_staff1_username(role, update_username, new_username):
        file_path = 'customer_staff.txt'
        updated_data = []
        Open file_path
        Read file_path as file:
        For staff_data in updated_data:
            If staff_data['username'] == updated_username:
                Staff_data ['username'] = new_username
            Break
Define update_staff_1_name(role, update_username, new_name)
    file_path = "customer_staff.txt"

Initialize updated_data as an empty list
Open file_path as file in read mode
    Read lines from file
    For each line in lines
        Split line by ', ' into staff_info
        Initialize staff_dict as an empty dictionary
        For each info in staff_info
            Split info by ': ' into key and value
            Assign stripped key and value to staff_dict
        Append staff_dict to updated_data

For each staff_data in updated_data
    If staff_data['username'] equals update_username
        Set staff_data['name'] to new_name
        Break

Open file_path as file in write mode
    For each staff_data in updated_data
        Write staff_data as a comma-separated string to file

Define update_customer_staff_1_password(role, update_username, new_password)
```

```
file_path = "customer_staff.txt"
```

```
Initialize updated_data as an empty list
```

```
Open file_path as file in read mode
```

```
    Read lines from file
```

```
    For each line in lines
```

```
        Split line by ',' into staff_info
```

```
        Initialize staff_dict as an empty dictionary
```

```
        For each info in staff_info
```

```
            Split info by ':' into key and value
```

```
            Assign stripped key and value to staff_dict
```

```
        Append staff_dict to updated_data
```

```
For each staff_data in updated_data
```

```
    If staff_data['username'] equals update_username
```

```
        Set staff_data['password'] to new_password
```

```
    Break
```

```
Open file_path as file in write mode
```

```
    For each staff_data in updated_data
```

```
        Write staff_data as a comma-separated string to file
```

```
Define update_manager_username(role, update_username, new_username)
```

```
    file_path = 'manager.txt'
```

```
Initialize updated_data as an empty list
```

```
Open file_path as file in read mode
```

```
    Read lines from file
```

```
    For each line in lines
```

```
        Split line by ',' into staff_info
```

```
        Initialize staff_dict as an empty dictionary
```

```
        For each info in staff_info
```

```
            Split info by ':' into key and value
```

```
            Assign stripped key and value to staff_dict
```

```
        Append staff_dict to updated_data
```

```
For each staff_data in updated_data
    If staff_data['username'] equals update_username
        Set staff_data['username'] to new_username
        Break
```

```
Open file_path as file in write mode
    For each staff_data in updated_data
        Write staff_data as a comma-separated string to file
```

```
Define update_manager_name(role, update_username, new_name)
    file_path = "manager.txt"
```

```
Initialize updated_data as an empty list
Open file_path as file in read mode
    Read lines from file
    For each line in lines
        Split line by ',' into staff_info
        Initialize staff_dict as an empty dictionary
        For each info in staff_info
            Split info by ':' into key and value
            Assign stripped key and value to staff_dict
        Append staff_dict to updated_data
```

```
For each staff_data in updated_data
    If staff_data['username'] equals update_username
        Set staff_data['name'] to new_name
        Break
```

```
Open file_path as file in write mode
    For each staff_data in updated_data
        Write staff_data as a comma-separated string to file
```

```
Define update_manager_password(role, update_username, new_password)
    file_path = "manager.txt"
```

```
Initialize updated_data as an empty list
```

```
Open file_path as file in read mode
    Read lines from file
    For each line in lines
        Split line by ',' into staff_info
        Initialize staff_dict as an empty dictionary
        For each info in staff_info
            Split info by ':' into key and value
            Assign stripped key and value to staff_dict
        Append staff_dict to updated_data

For each staff_data in updated_data
    If staff_data['username'] equals update_username
        Set staff_data['password'] to new_password
        Break

Open file_path as file in write mode
    For each staff_data in updated_data
        Write staff_data as a comma-separated string to file

Define update_carstaff_username(role, update_username, new_username)
    file_path = 'car_service_staff.txt'

    Initialize updated_data as an empty list
    Open file_path as file in read mode
        Read lines from file
        For each line in lines
            Split line by ',' into staff_info
            Initialize staff_dict as an empty dictionary
            For each info in staff_info
                Split info by ':' into key and value
                Assign stripped key and value to staff_dict
            Append staff_dict to updated_data

    For each staff_data in updated_data
        If staff_data['username'] equals update_username
            Set staff_data['username'] to new_username
```

Break

Open file_path as file in write mode

For each staff_data in updated_data

Write staff_data as a comma-separated string to file

Define update_carstaff_name(role, update_username, new_name)

file_path = 'car_service_staff.txt'

Initialize updated_data as an empty list

Open file_path as file in read mode

Read lines from file

For each line in lines

Split line by ',' into staff_info

Initialize staff_dict as an empty dictionary

For each info in staff_info

Split info by ':' into key and value

Assign stripped key and value to staff_dict

Append staff_dict to updated_data

For each staff_data in updated_data

If staff_data['username'] equals update_username

Set staff_data['name'] to new_name

Break

Open file_path as file in write mode

For each staff_data in updated_data

Write staff_data as a comma-separated string to file

def update_carstaff_password(role, update_username, new_password):

Define the file path based on the role

file_path = 'car_service_staff.txt'

Read data from the text file

updated_data = []

with open(file_path, 'r') as file:

lines = file.readlines()

```

for line in lines:
    staff_info = line.strip().split(',')
    staff_dict = {}
    for info in staff_info:
        key, value = info.split(': ')
        staff_dict[key.strip()] = value.strip()
    updated_data.append(staff_dict)

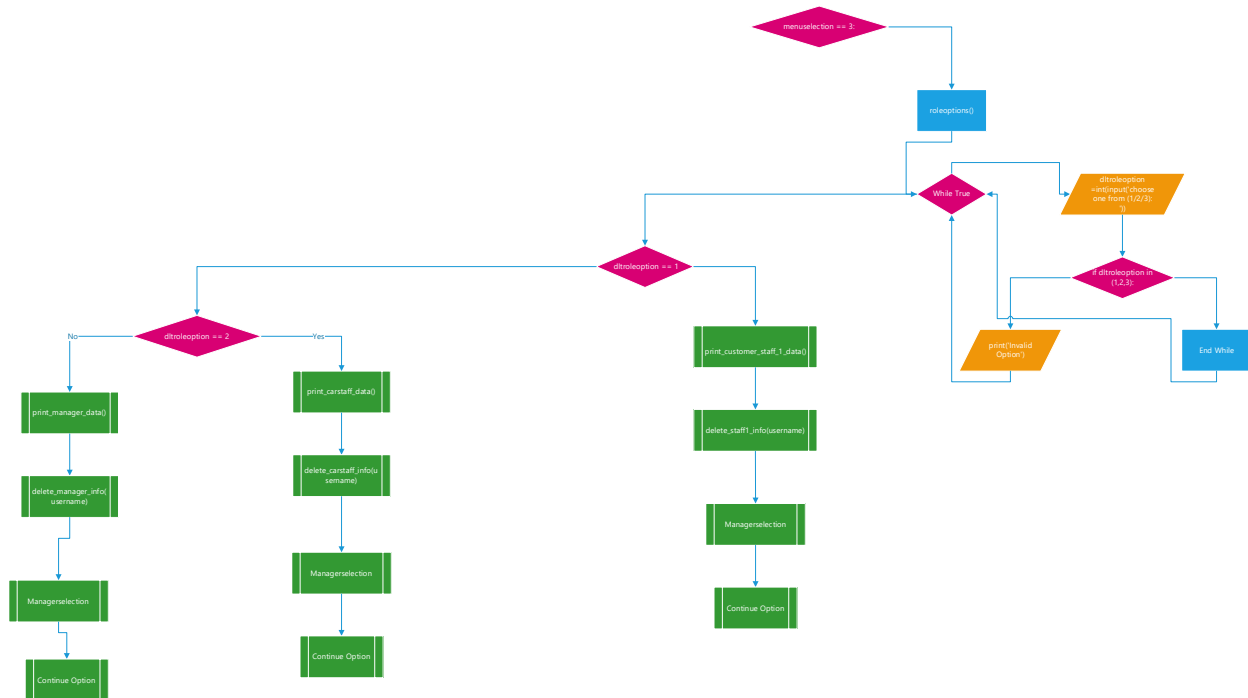
# Update the password in the data
for staff_data in updated_data:
    if staff_data['username'] == update_username:
        staff_data['password'] = new_password
        break

    # Write the updated data back to the text file
with open(file_path, 'w') as file:
    for staff_data in updated_data:
        file.write(', '.join([f'{key}: {value}' for key, value in staff_data.items()]) + '\n')

def print_customer_staff_1_data():
    with open('customer_staff.txt', 'r') as file:
        print()
        print("Customer Service Staff List:")
        print()
        lines = file.readlines()
        for line in lines:
            if line.strip(): # Check if the line is not empty
                data = line.strip().split(',')
                for item in data:
                    key, value = item.split(': ')
                    print(f'{key}: {value}')

```

3.1.4 Delete Staff



```

def delete_staff1_info(username):
    found = False
    with open('customer_staff.txt', 'r') as file:
        lines = file.readlines()

    with open('customer_staff.txt', 'w') as file:
        for line in lines:
            if f'username: {username}' in line: # Adjusted search criteria
                print(f'Staff with username '{username}' deleted successfully.')
                found = True
            else:
                file.write(line)

    if not found:
        print("Staff not found.")

def print_carstaff_data():
    with open('car_service_staff.txt', 'r') as file:
        print()
        print("Car Service Staff List:")
  
```

```
print()
lines = file.readlines()
for line in lines:
    if line.strip(): # Check if the line is not empty
        data = line.strip().split(',')
        for item in data:
            key, value = item.split(': ')
            print(f'{key}: {value}')
        print()

def delete_carstaff_info(username):
    found = False
    with open('car_service_staff.txt', 'r') as file:
        lines = file.readlines()

    with open('car_service_staff.txt', 'w') as file:
        for line in lines:
            if f'username: {username}' in line: # Adjusted search criteria
                print(f'Staff with username '{username}' deleted successfully.")
                found = True
            else:
                file.write(line)

    if not found:
        print("Staff not found.")

def print_manager_data():
    with open('manager.txt', 'r') as file:
        print()
        print("Manager List:")
        print()
        lines = file.readlines()
        for line in lines:
            if line.strip(): # Check if the line is not empty
                data = line.strip().split(',')
                for item in data:
                    key, value = item.split(': ')
```



```

        print(f'{key}: {value}')
    print()

```

```

def delete_manager_info(username):
    found = False
    with open('Manager.txt', 'r') as file:
        lines = file.readlines()

    with open('Manager.txt', 'w') as file:
        for line in lines:
            if f'username: {username}' in line: # Adjusted search criteria
                print(f'Staff with username '{username}' deleted successfully.")
                found = True
            else:
                file.write(line)

    if not found:
        print("Staff not found.")
    Else if menuselection is 3:
        Display role options
        While True:
            Input dltroleoption
            If dltroleoption is 1, 2, or 3:
                Break
            Else:
                Print "Invalid Option"

        If dltroleoption is 1:
            Print_customer_staff_1_data()
            Input username_to_delete
            Call delete_staff1_info with username_to_delete

        Else if dltroleoption is 2:
            Print_carstaff_data()
            Input username_to_delete
            Call delete_carstaff_info with username_to_delete

```

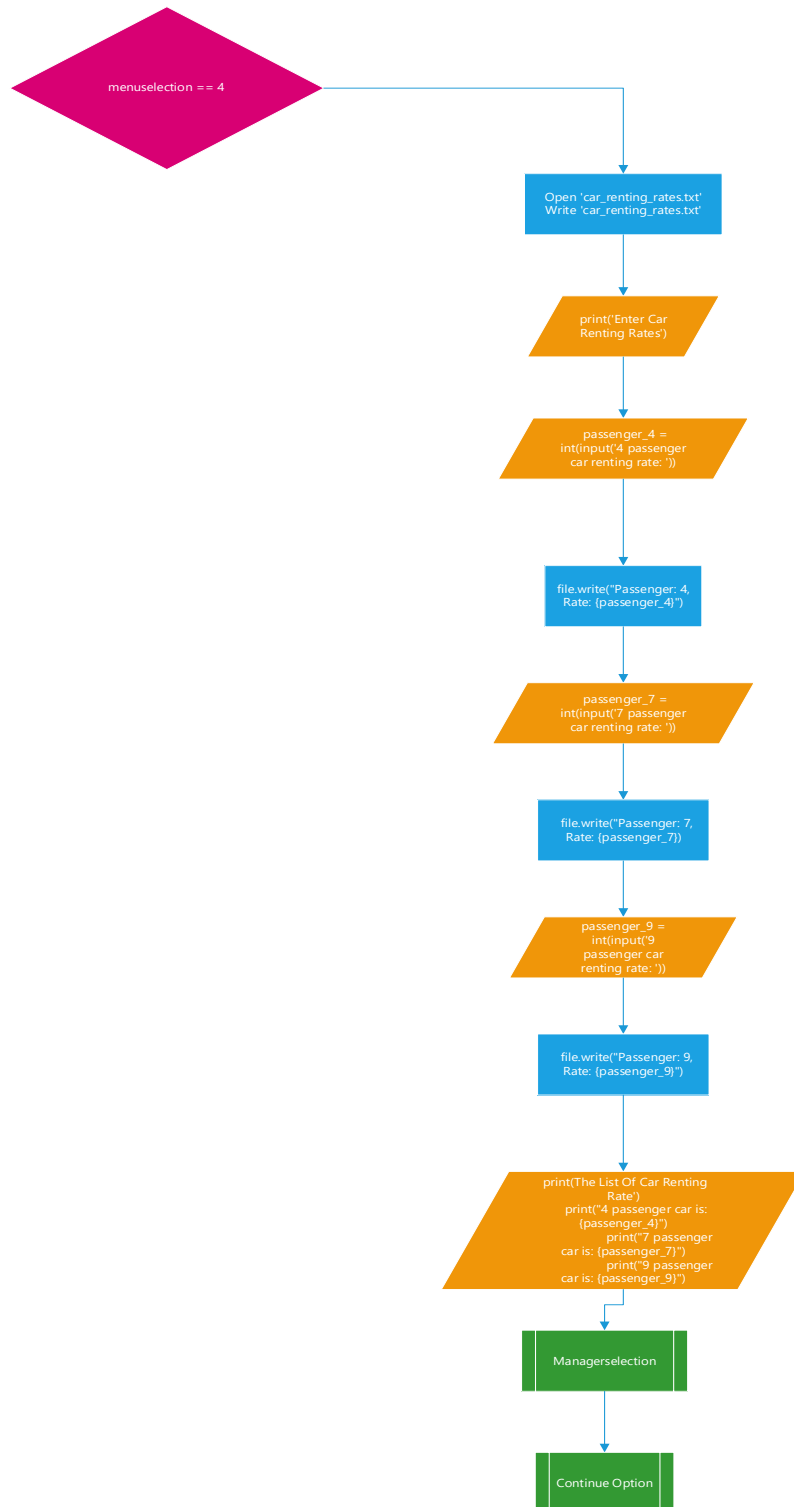
Else if dltroleoption is 3:

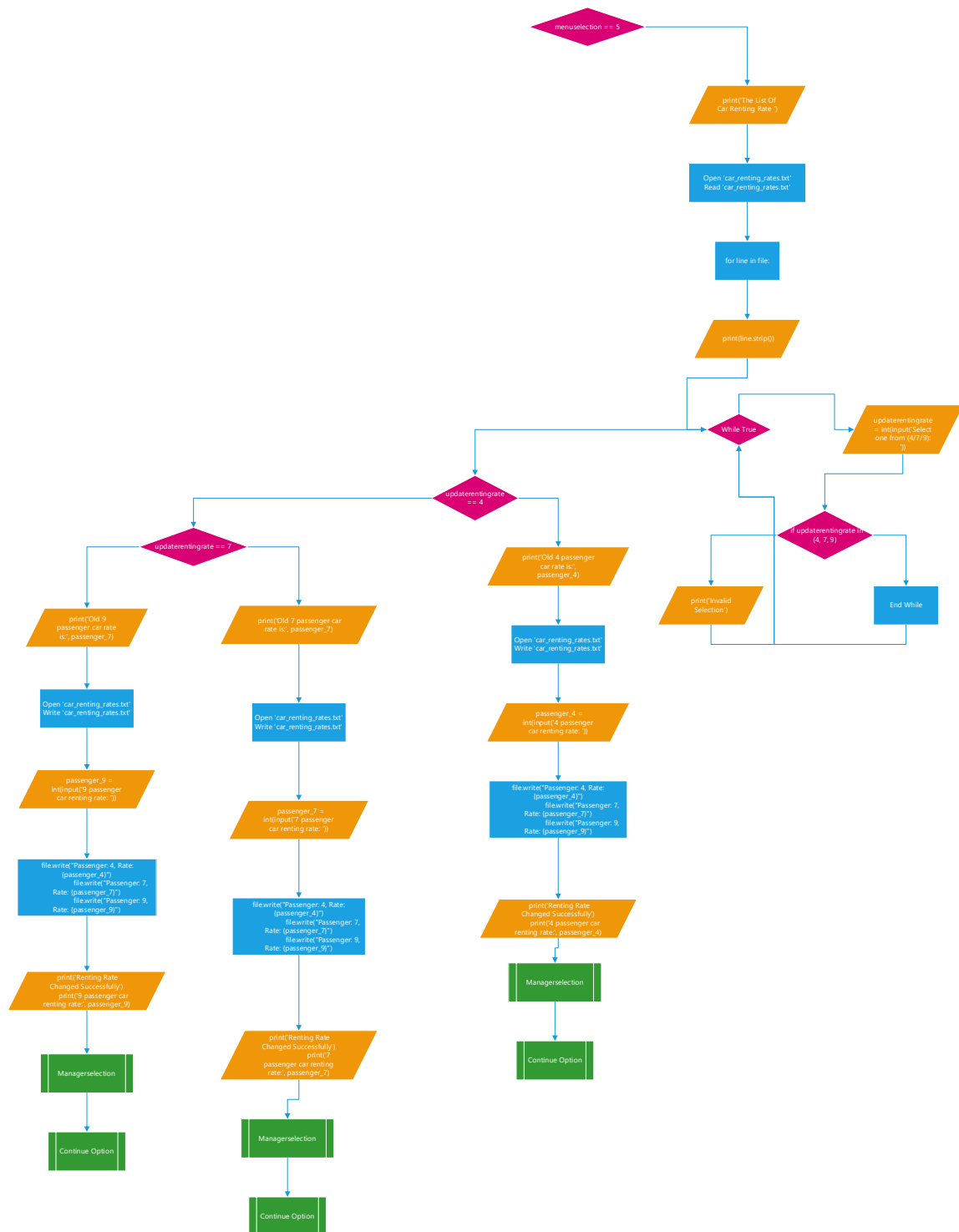
 Print_manager_data()

 Input username_to_delete

 Call delete_manager_info with username_to_delete

3.1.5 Set and update car renting rate





Else if menuselection is 4:

With open('car_renting_rates.txt', 'w') as file:

Print "Enter Car Renting Rates"

Input passenger_4

```
Write "Passenger: 4, Rate: {passenger_4}" to file
Input passenger_7
Write "Passenger: 7, Rate: {passenger_7}" to file
Input passenger_9
Write "Passenger: 9, Rate: {passenger_9}" to file
Print "The List Of Car Renting Rate"
Print "4 passenger car is: {passenger_4}"
Print "7 passenger car is: {passenger_7}"
Print "9 passenger car is: {passenger_9}"
```

Else if menuselection is 5:

```
Print "The List Of Car Renting Rate"
With open('car_renting_rates.txt', 'r') as file:
    For each line in file:
        Print line.strip()
```

While True:

```
Input updatere rentingrate
If updatere rentingrate is 4, 7, or 9:
    Break
Else:
    Print "Invalid Selection"
```

If updatere rentingrate is 4:

```
Print "Old 4 passenger car rate is:", passenger_4
With open('car_renting_rates.txt', 'w') as file:
    Input passenger_4
    Write "Passenger: 4, Rate: {passenger_4}" to file
    Write "Passenger: 7, Rate: {passenger_7}" to file
    Write "Passenger: 9, Rate: {passenger_9}" to file
Print "Renting Rate Changed Successfully"
Print "4 passenger car renting rate:", passenger_4
```

If updatere rentingrate is 7:

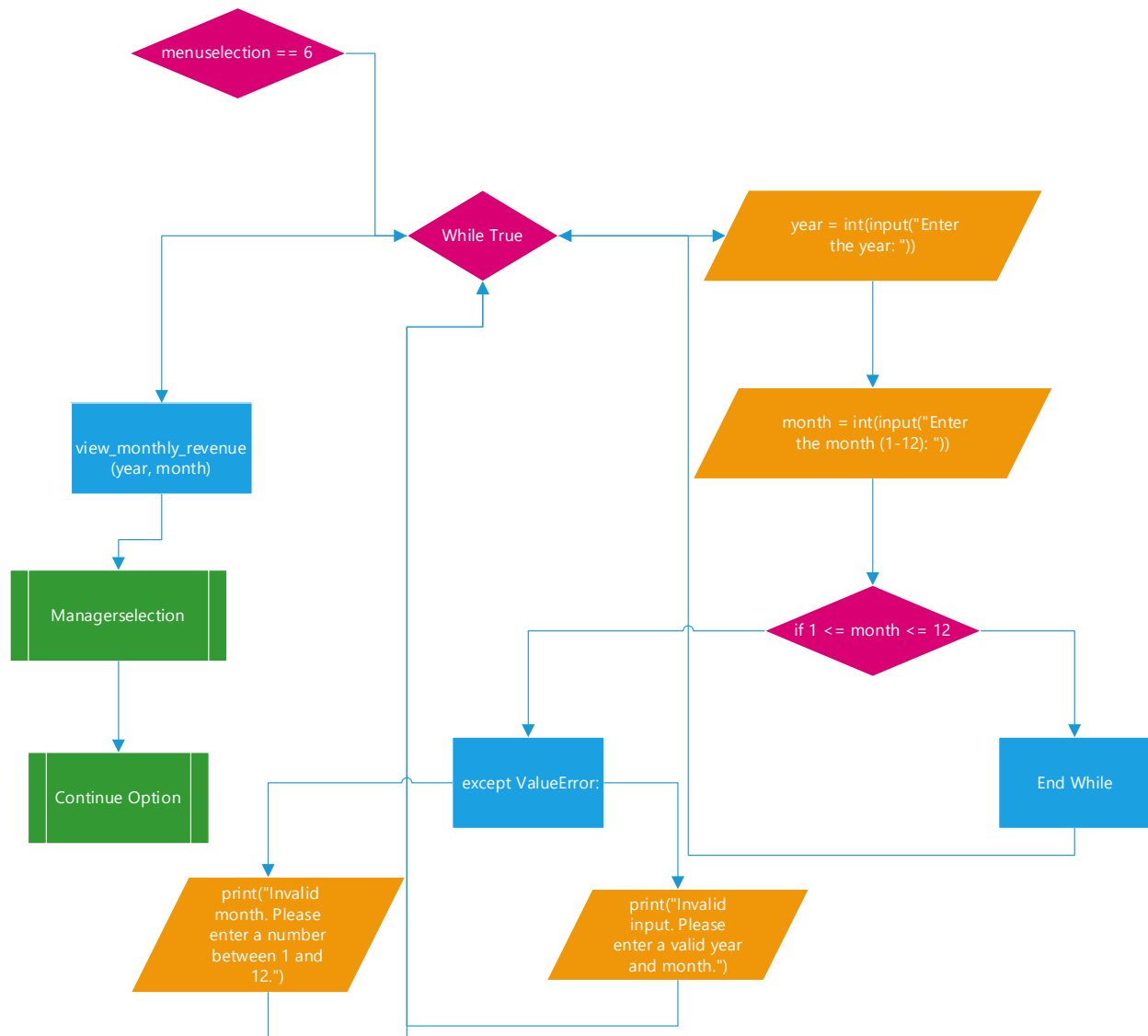
```
Print "Old 7 passenger car rate is:", passenger_7
With open('car_renting_rates.txt', 'w') as file:
    Input passenger_7
```

```
Write "Passenger: 4, Rate: {passenger_4}" to file
Write "Passenger: 7, Rate: {passenger_7}" to file
Write "Passenger: 9, Rate: {passenger_9}" to file
Print "Renting Rate Changed Successfully"
Print "7 passenger car renting rate:", passenger_7
```

If updatere rentingrate is 9:

```
Print "Old 9 passenger car rate is:", passenger_7
With open('car_renting_rates.txt', 'w') as file:
    Input passenger_9
    Write "Passenger: 4, Rate: {passenger_4}" to file
    Write "Passenger: 7, Rate: {passenger_7}" to file
    Write "Passenger: 9, Rate: {passenger_9}" to file
Print "Renting Rate Changed Successfully"
Print "9 passenger car renting rate:", passenger_9
```

3.1.6 View monthly revenue report



Else if menuselection is 6:

While True:

Try:

Input year

Input month

If month is between 1 and 12:

Break

Else:

Print "Invalid month. Please enter a number between 1 and 12."

Except ValueError:

Print "Invalid input. Please enter a valid year and month."

View_monthly_revenue(year, month)

While True:

 Input choice

 If choice is 'yes' or 'no':

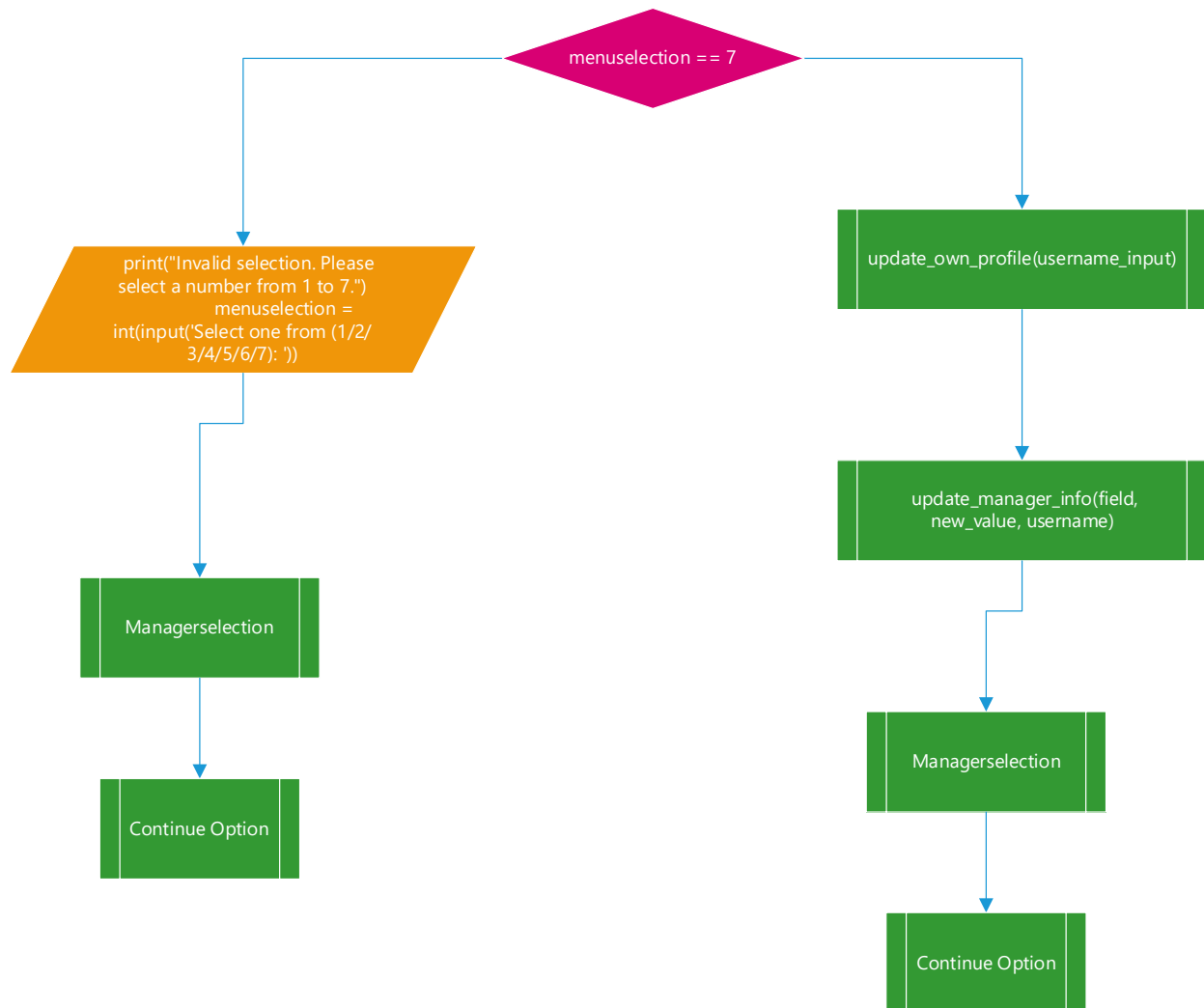
 Break

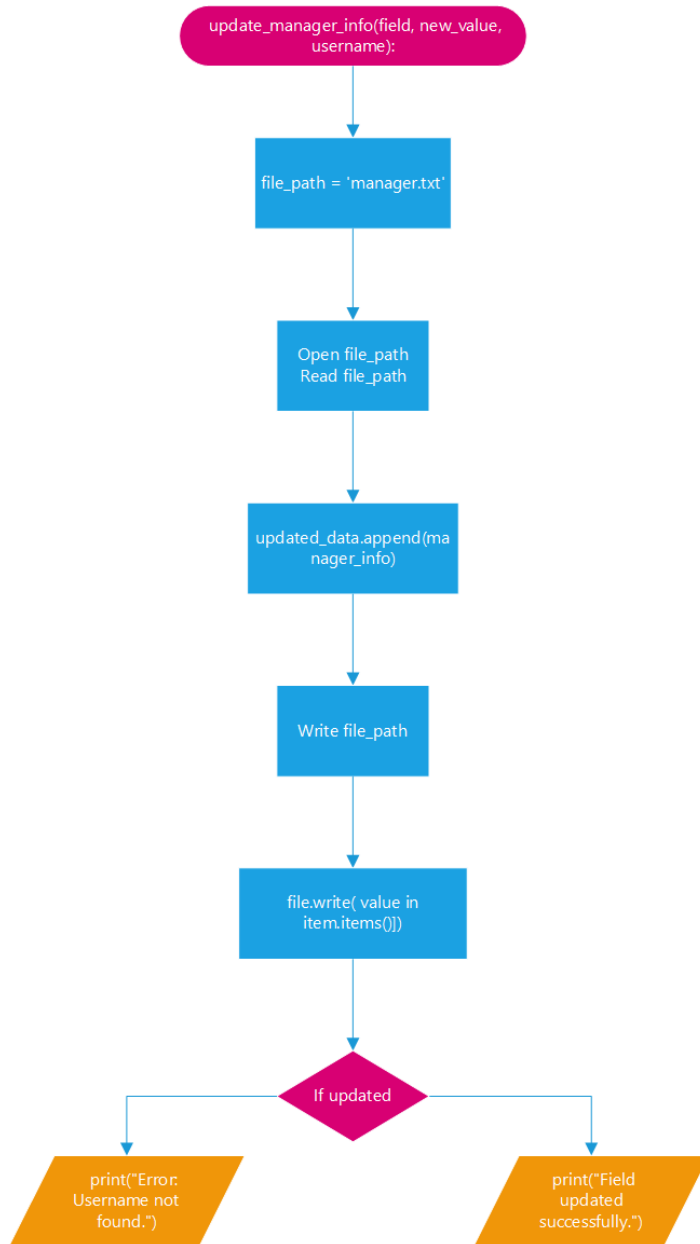
 Else:

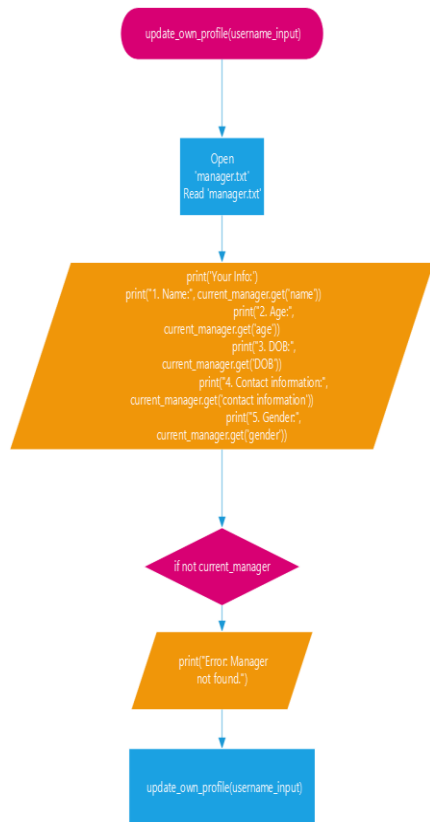
 Print "Enter yes or no only"

If choice is not 'yes':

3.1.7 Update profile







Else if menuselection is 7:

Update_own_profile(username_input)

Input managerinfoselection

While True:

If managerinfoselection is 1, 2, 3, 4, or 5:

Break

Else:

Print "Invalid Selection"

Input managerinfoselection

If managerinfoselection is 1:

Input newinfofname

Call update_manager_info with 'name', newinfofname, username_input

If managerinfoselection is 2:

Input newinfoage

Call update_manager_info with 'age', newinfoage, username_input

If managerinfoselection is 3:

Input newinfoDOB

Call update_manager_info with 'DOB', newinfoDOB, username_input

If managerinfoselection is 4:

Input newinfoCI

Call update_manager_info with 'contact information', newinfoCI, username_input

If managerinfoselection is 5:

Input newinfogender

Call update_manager_info with 'gender', newinfogender, username_input

Else:

Print "Invalid selection. Please select a number from 1 to 7."

While True:

Input choice

If choice is 'yes' or 'no':

Break

Else:

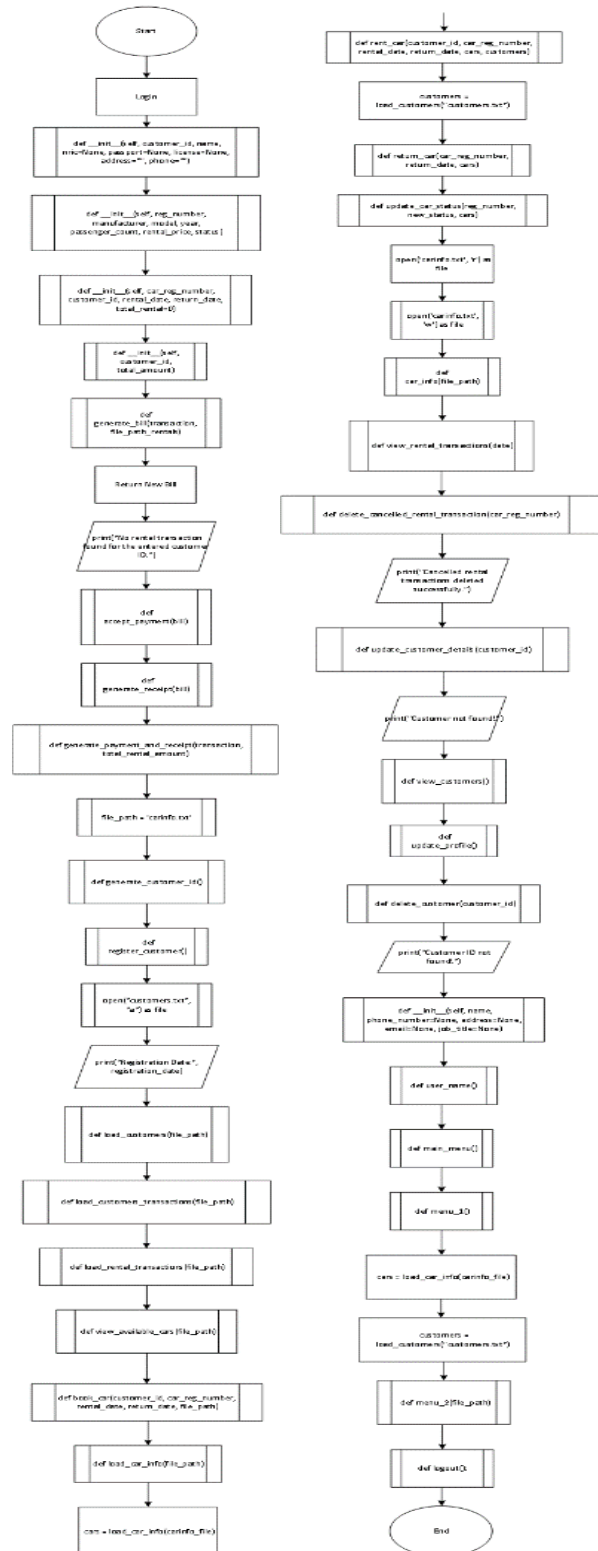
Print "Enter yes or no only"

If choice is not 'yes':

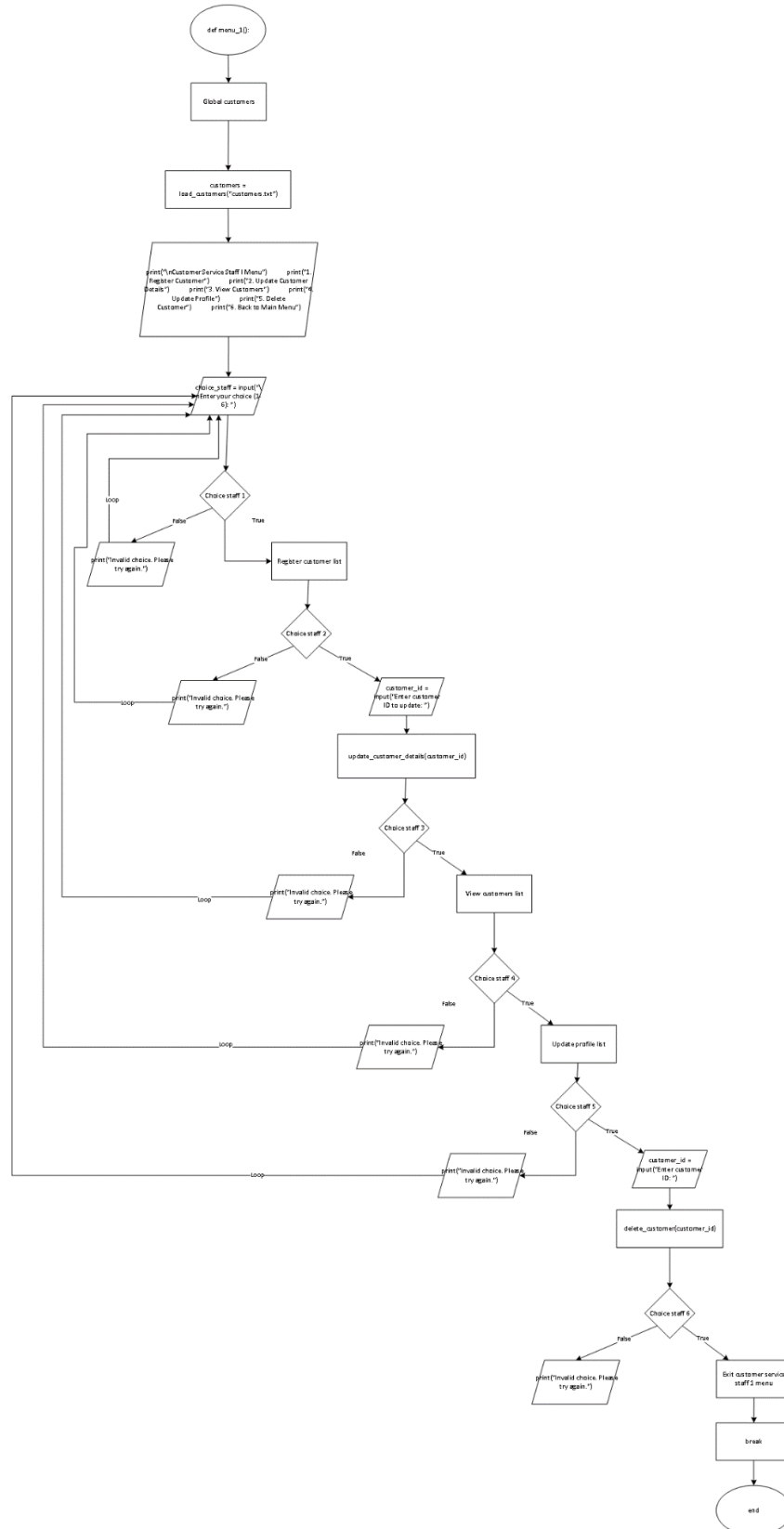
End

3.2 Customer Service Staff 1

3.2.1 Main Menu



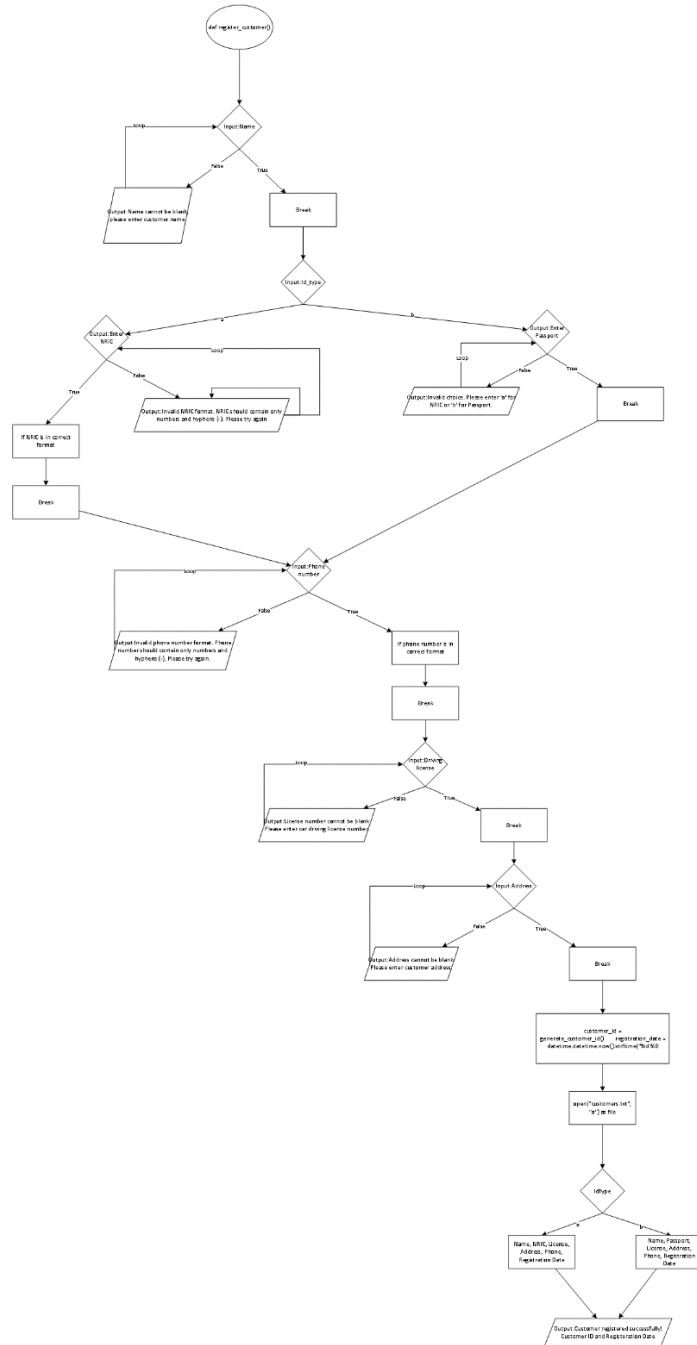
3.2.2 Menu 1



Start

```
Define menu_1
    Set customers as global
    Load existing customers from "customers.txt"
    Set profile_file_path as "customer_staff_profile.txt"
    While True
        Print Customer Service Staff I menu options
        Ask for choice (1-6)
        If choice is not in ('1', '2', '3', '4', '5', '6')
            Print "Invalid choice. Please try again."
            Continue
        If choice is '1'
            Register customer
        ElseIf choice is '2'
            Ask for customer ID to update
            Update customer details
        ElseIf choice is '3'
            View customers from "customers.txt"
        ElseIf choice is '4'
            Update profile using profile_file_path
        ElseIf choice is '5'
            Ask for customer ID
            Delete customer
        ElseIf choice is '6'
            Break
```

3.2.3 Register Customer



define function register_customer

repeatedly prompt user to input customer details until valid inputs are provided for each field

generate unique customer ID using generate_customer_id function

record current date as registration_date

write customer information to "customers.txt" file

print confirmation message with customer ID and registration date


```

graph TD
    Start([Start]) --> ReadGraph[Read graph G]
    ReadGraph --> IsTree{Is graph G a tree?}
    IsTree -- Yes --> OutputTree[Output result: 1]
    IsTree -- No --> FindMinPhases[Find minimum number of phases for graph G]
    FindMinPhases --> WriteMinPhases[Write min_phases to file min_phases.txt]
    WriteMinPhases --> ReadMinPhases[Read min_phases from file min_phases.txt]
    ReadMinPhases --> IsMinPhasesEqual{Is min_phases equal to number of vertices?}
    IsMinPhasesEqual -- Yes --> OutputResult[Output result: min_phases]
    IsMinPhasesEqual -- No --> FindMinPhasesAgain[Find minimum number of phases for graph G again]
    FindMinPhasesAgain --> WriteMinPhasesAgain[Write min_phases to file min_phases.txt]
    WriteMinPhasesAgain --> ReadMinPhasesAgain[Read min_phases from file min_phases.txt]
    ReadMinPhasesAgain --> IsMinPhasesEqualAgain{Is min_phases equal to number of vertices?}
    IsMinPhasesEqualAgain -- Yes --> OutputResultAgain[Output result: min_phases]
    IsMinPhasesEqualAgain -- No --> FindMinPhasesAgain
    
```

Define update_customer_details

Parameters: customer_id

Set customers as global

Remove extra whitespace from customer_id

For each customer in customers

Extract customer ID from customer object without any prefix

If extracted customer ID equals customer_id

Print customer details

Define available_fields as dictionary of fields to update

Define allowed_choices as list of allowed choices

Ask for fields to update

Split fields_to_update into list of fields

Set update_successful as False

For each field in fields

Remove extra whitespace from field

If field in allowed_choices

If field is 'a' (phone)

Ask for new phone number

If new phone number contains only digits and hyphens

Update customer phone number

Set update_successful as True

Else

Print "Invalid phone number format. Please use only numbers and hyphens."

ElseIf field is 'b' (address)

Ask for new address

Update customer address

Set update_successful as True

ElseIf field is 'c' (license)

Ask for new license number

Update customer license number

Set update_successful as True

ElseIf field is 'd' (passport)

If customer is NRIC

Print "NRIC customer cannot update passport."

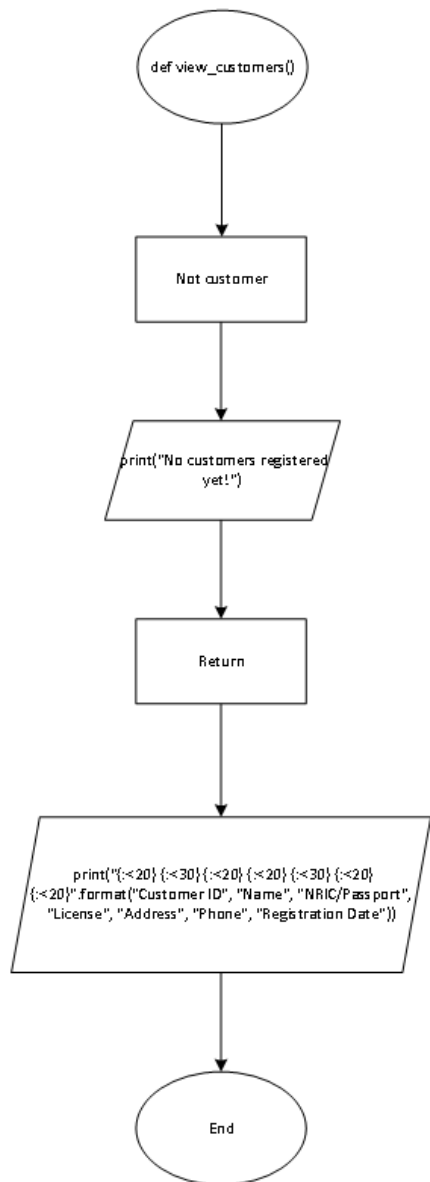
Else

Ask for new passport number

Update customer passport number

```
        Set update_successful as True
    Else
        Print "Invalid field: {field}"
    If update_successful
        Print "Customer details updated successfully!"
        Open "customers.txt" in write mode as file
        For each cust in customers
            Write customer details in specified format to file
        Return
    Print "Customer not found!"
```

3.2.5 View List of Registered Customers



Define view_customers

Parameters: file_path

Load customer details from file_path into customers

If customers is empty

Print "No customers registered yet!"

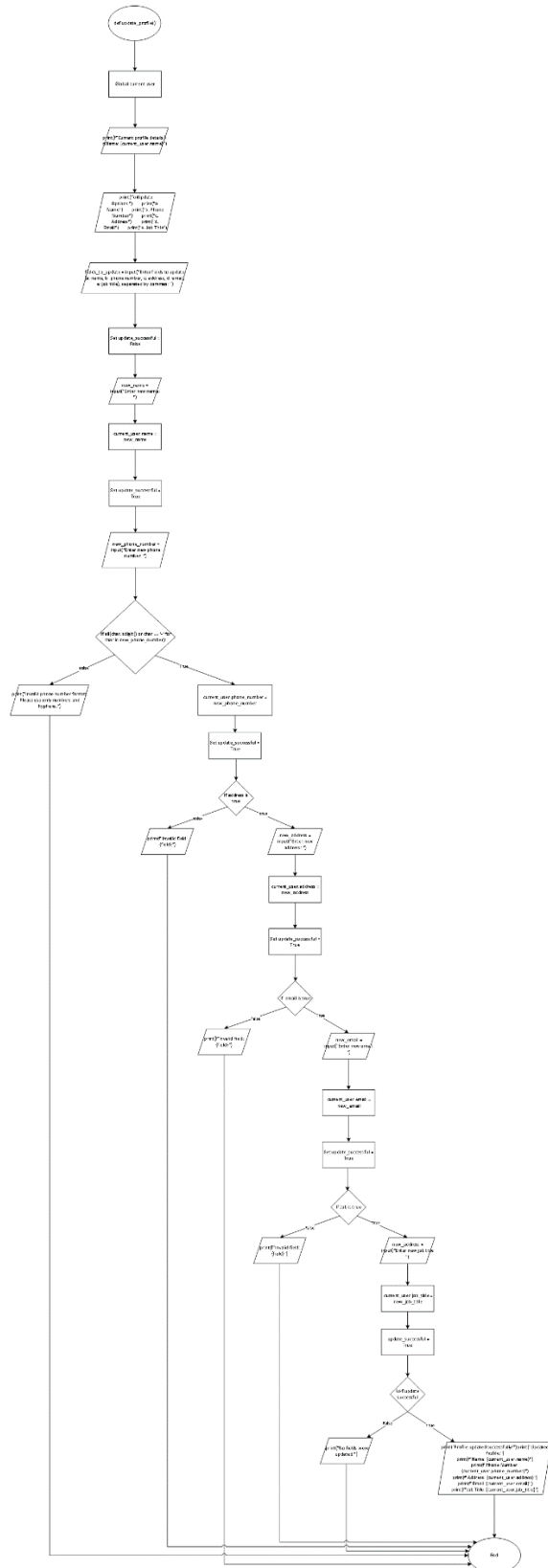
Return

Print formatted table header

For each customer in customers

Print formatted customer details

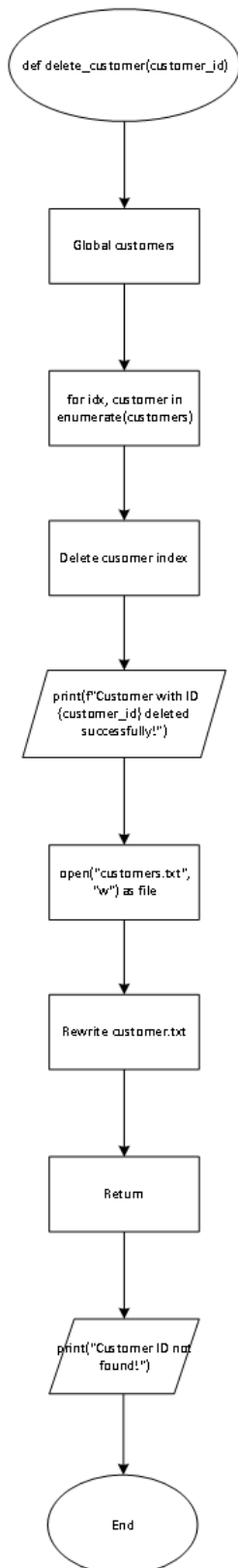
3.2.6 Updated Profile



```
Define update_profile
  Parameters: file_path
  Set current_user as global
  If profile file exists and is not empty
    Open file_path in read mode as file
    Read line from file and strip
    If line exists
      Split line into data by pipe and strip each item
      If length of data is 5
        Set current_user as UserProfile object with unpacked data
      Else
        Print "Invalid data format in profile file."
      Return
  Else
    Print "Please enter your details."
    Ask for name, phone number, address, email, job title
    Set current_user as UserProfile with entered details
    Open file_path in write mode as file
    Write current_user details to file
    Print "Profile created successfully for {current_user.name}!"
    Return
  Print current_user details
  Print "Update Options:"
  Print options for fields to update
  Define available_fields as dictionary of fields to update
  Define allowed_choices as list of allowed choices
  Ask for fields to update
  Split fields_to_update into list of fields
  Set update_successful as False
  For each field in fields
    Remove extra whitespace from field
    If field in allowed_choices
      If field is 'a' (name)
        Ask for new name
        Update current_user name
        Set update_successful as True
      ElseIf field is 'b' (phone number)
```

```
    Ask for new phone number
    If new phone number contains only digits and hyphens
        Update current_user phone number
        Set update_successful as True
    Else
        Print "Invalid phone number format. Please use only numbers and hyphens."
    ElseIf field is 'c' (address)
        Ask for new address
        Update current_user address
        Set update_successful as True
    ElseIf field is 'd' (email)
        Ask for new email
        Update current_user email
        Set update_successful as True
    ElseIf field is 'e' (job title)
        Ask for new job title
        Update current_user job title
        Set update_successful as True
    Else
        Print "Invalid field: {field}"
If update_successful
    Print "Profile updated successfully!"
    Print updated current_user details
    Open file_path in write mode as file
    Write current_user details to file
Else
    Print "No fields were updated."
```

3.2.7 Delete Customers who have no rental transaction recorded

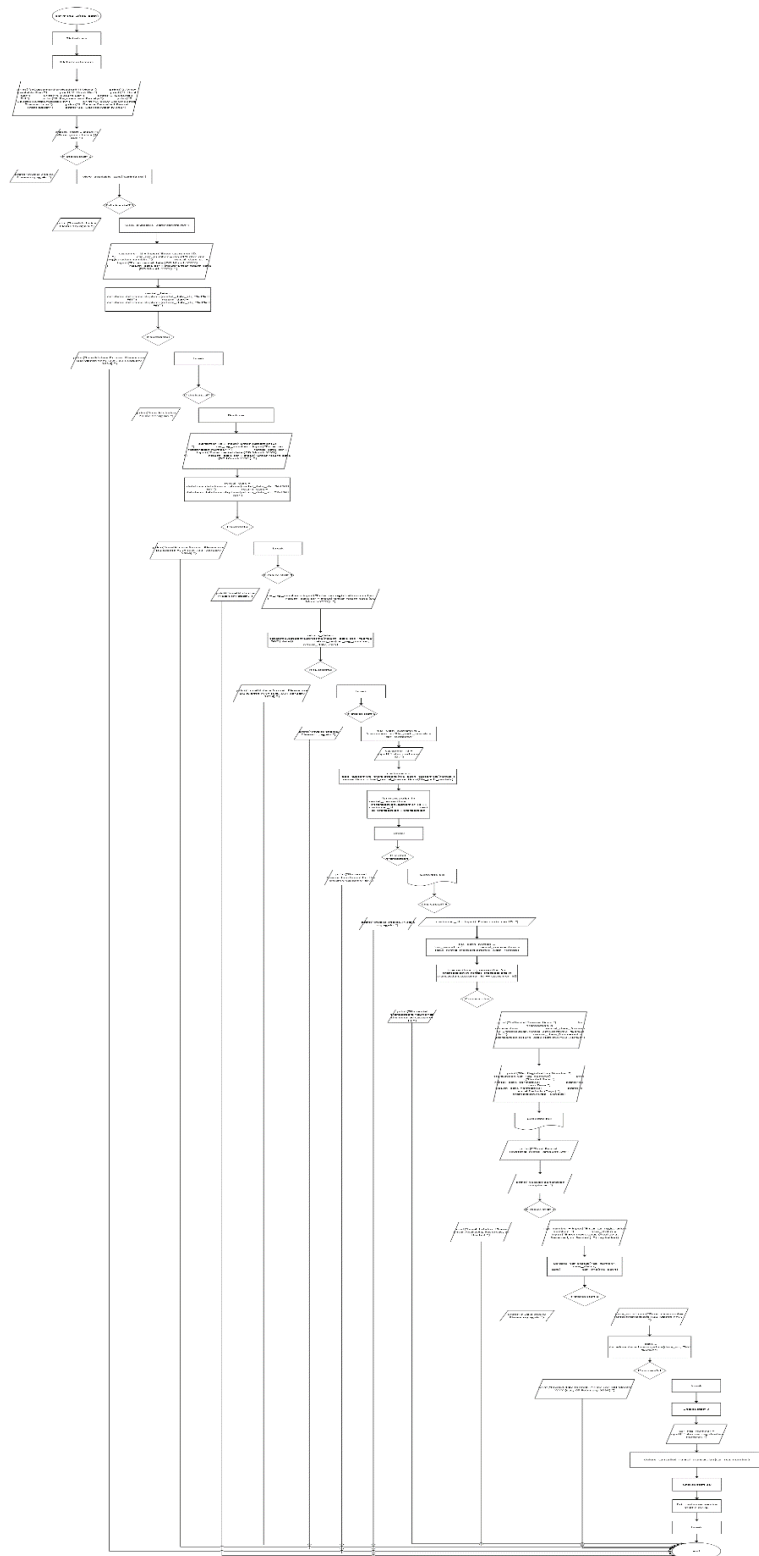



```
Define delete_customer
    Parameters: customer_id
    Set customers as global
    For each idx, customer in enumerate(customers)
        If customer ID equals customer_id (after stripping whitespace)
            Delete customer at idx from customers
            Print "Customer with ID {customer_id} deleted successfully!"
        For each i, cust in enumerate(customers, start=1)
            Update customer ID with leading zeros
            Open "customers.txt" in write mode as file
            For each cust in customers
                Write customer details in specified format to file
            Return
    Print "Customer ID not found!"

End
```

3.3 Customer Service Staff 2

3.3.1 Menu 2



Start

Define menu_2

Parameters: file_path

Set cars and customers as global

While True

Print Customer Service Staff II menu options

Ask for choice (1-10)

If choice is not in ('1', '2', '3', '4', '5', '6', '7', '8', '9', '10')

Print "Invalid choice. Please try again."

Continue

If choice is '1'

View available cars from 'carinfo.txt'

ElseIf choice is '2'

While True

Ask for customer ID

Ask for car registration number

Ask for rental date

Ask for return date

Try to convert rental and return date strings to datetime objects

If successful, break loop

Else, print error message

Call book_car function with provided inputs

ElseIf choice is '3'

While True

Ask for customer ID

Ask for car registration number

Ask for rental date

Ask for return date

Try to convert rental and return date strings to datetime objects

If successful, call rent_car function with provided inputs

Break loop after successful rental

Else, print error message

ElseIf choice is '4'

Ask for car registration number

Ask for return date

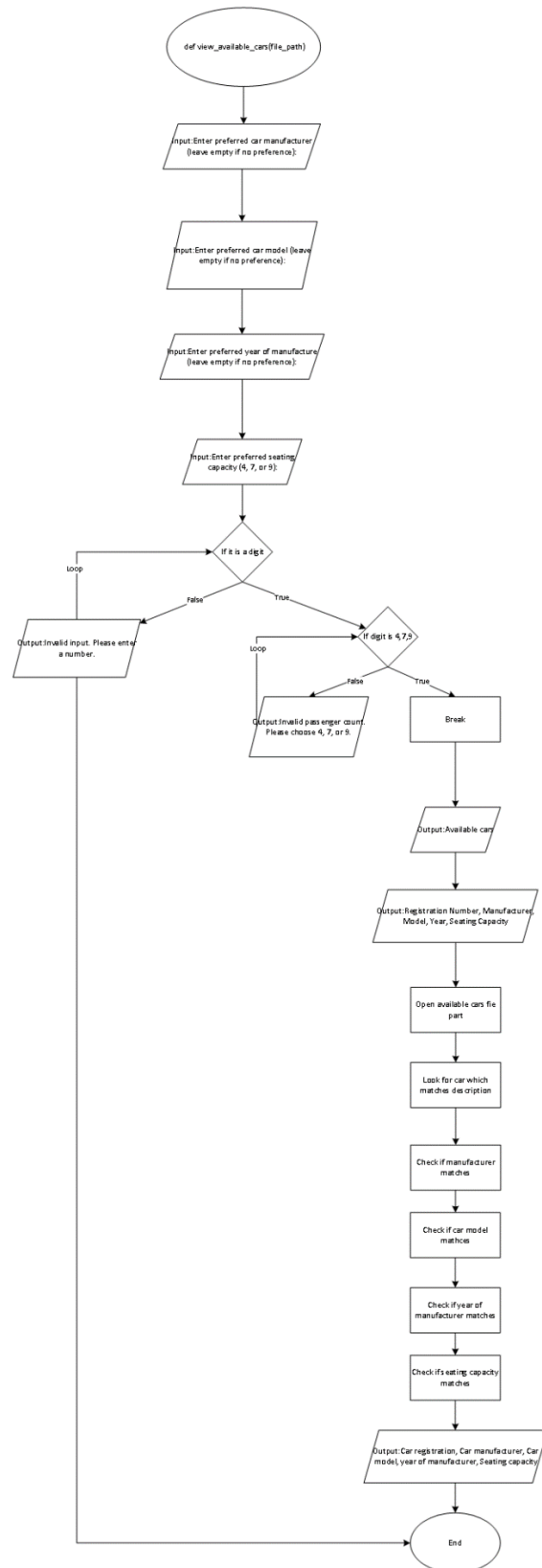
Try to convert return date string to datetime object

If successful, call return_car function with provided inputs

```
    Else, print error message
ElseIf choice is '5'
    Set file_path_customers as 'customers.txt'
    Set file_path_rentals as 'car_rental.txt'
    Ask for customer ID
    Load customer data
    Load rental transactions
    Initialize rental_transaction as None
    For each transaction in rental_transactions
        If transaction's customer ID matches entered customer ID
            Set rental_transaction as transaction
            Break loop
    If rental_transaction exists
        Call generate_bill with rental_transaction and file_path_rentals
    Else, print "No rental transaction found for the entered customer ID."
ElseIf choice is '6'
    Ask for customer ID
    Load rental transactions from 'car_rental.txt'
    Initialize transactions as list of transactions associated with entered customer ID
    If no transactions found
        Print "No rental transactions found for the entered customer ID."
    Else
        Print "Rental Transactions:"
        For each transaction in transactions
            Format rental_date and return_date
            Print transaction details
            Generate bill for transaction
            If bill is generated successfully
                Extract total_rental_amount from bill
                Print total_rental_amount
                Generate payment and receipt for transaction
            Print "Receipt generation completed."
ElseIf choice is '7'
    Ask for car registration number
    Ask for new status (Available, Reserved, or Rented), capitalize it
    If new status is not in ['Available', 'Reserved', 'Rented']
        Print "Invalid status. Please enter Available, Reserved, or Rented."
```

```
    Else
        Call update_car_status with provided inputs
        Call car_info with file_path
    ElseIf choice is '8'
        While True
            Ask for date to view rental transactions
            Try to parse date string to datetime object
            If successful, break loop
            Else, print error message
        Call view_rental_transactions with parsed date
    ElseIf choice is '9'
        Ask for car registration number
        Call delete_cancelled_rental_transaction with provided car registration number
    ElseIf choice is '10'
        Break
```

3.3.2 Check Availability



```
define function view_available_cars with parameter file_path
    prompt user to input preferences for car manufacturer, model, year of manufacture, and seating capacity
    iterate over each line in file_path
        split line by ',' and create car_info dictionary
        if car is available and matches input preferences (or no preferences specified)
            print car details including registration number, manufacturer, model, year, and seating capacity
```

3.3.3 Accept and Record Rental Details



Define rent_car

Parameters: customer_id, car_reg_number, rental_date, return_date, cars, customers

Set customer_found as False

Loop over customers

 If customer.customer_id equals customer_id

 Set customer_found as True

 Break

If customer_found is False

 Print "Customer not found."

 Return False

Set car_found as False

Loop over cars

 If car.reg_number equals car_reg_number

 Set car_found as True

 If car.status is "Available" or "Reserved"

 Calculate rental_periods as (return_date - rental_date).days

 Set rental_price_per_day as car.rental_price

 Calculate total_rental as rental_periods * rental_price_per_day

 Format rental_date and return_date as strings

 Print "Car rented successfully!"

 Print "Rental Date:", rental_date_str

 Print "Return Date:", return_date_str

 Print "Rental Periods (Days):", rental_periods, "days"

 Print "Total Rental: RM{total_rental:.2f}"

 Write rental information to "car_rental.txt" file

 Return True

 Else

 Print "Car is not available for rent."

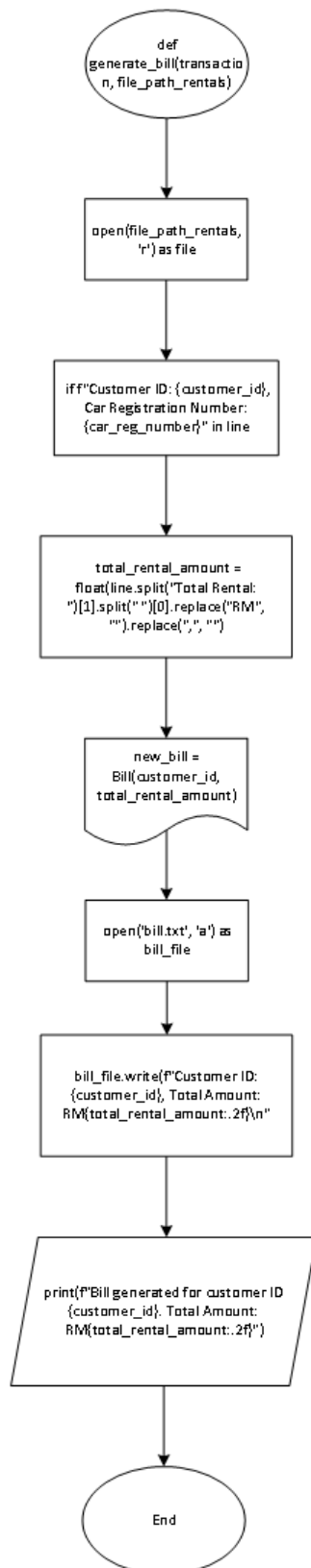
 Break

If car_found is False

 Print "Car not found."

Return False

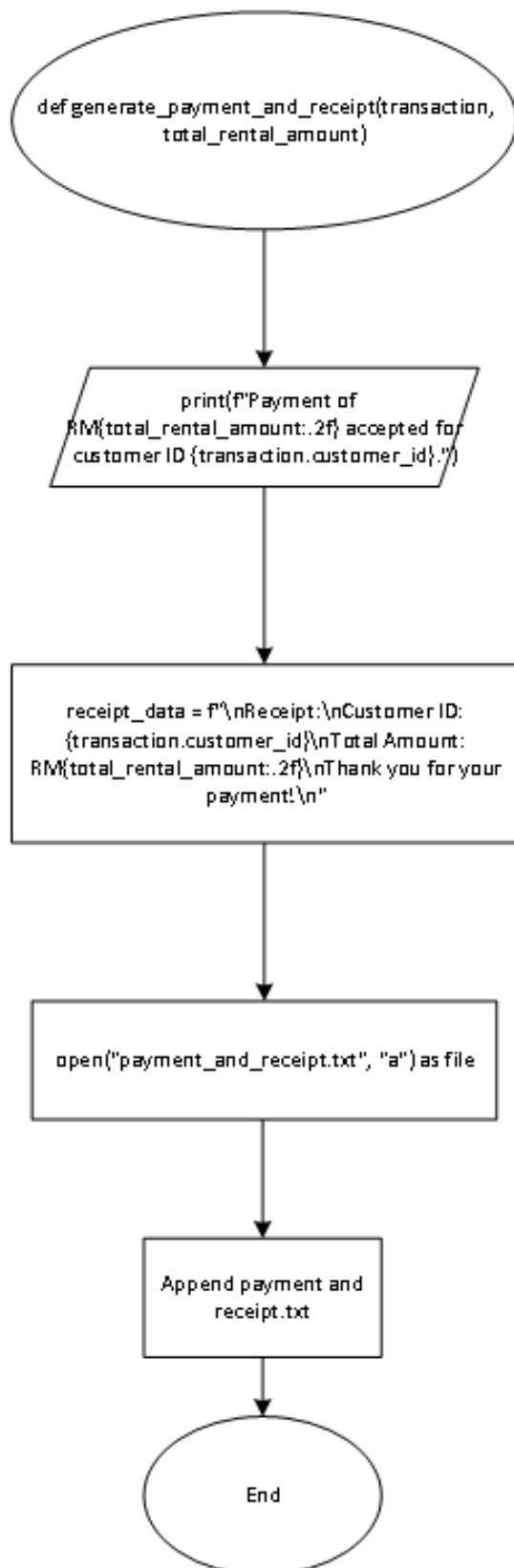
3.3.4 Generate Bills



```
def generate_bill(transaction, file_path_rentals):
    customer_id = transaction.customer_id
    car_reg_number = transaction.car_reg_number
    with open(file_path_rentals, 'r') as file:
        for line in file:
            if line contains customer_id and car_reg_number:
                total_rental_amount = extract_total_rental_amount(line)
                new_bill = Bill(customer_id, total_rental_amount)
                with open('bill.txt', 'a') as bill_file:
                    bill_file.write(new_bill.__str__() + '\n')
                print(f"Bill generated for customer ID {customer_id}. Total Amount: RM{total_rental_amount:.2f}")
    return new_bill

print("No rental transaction found for the entered customer ID.")
return None
```

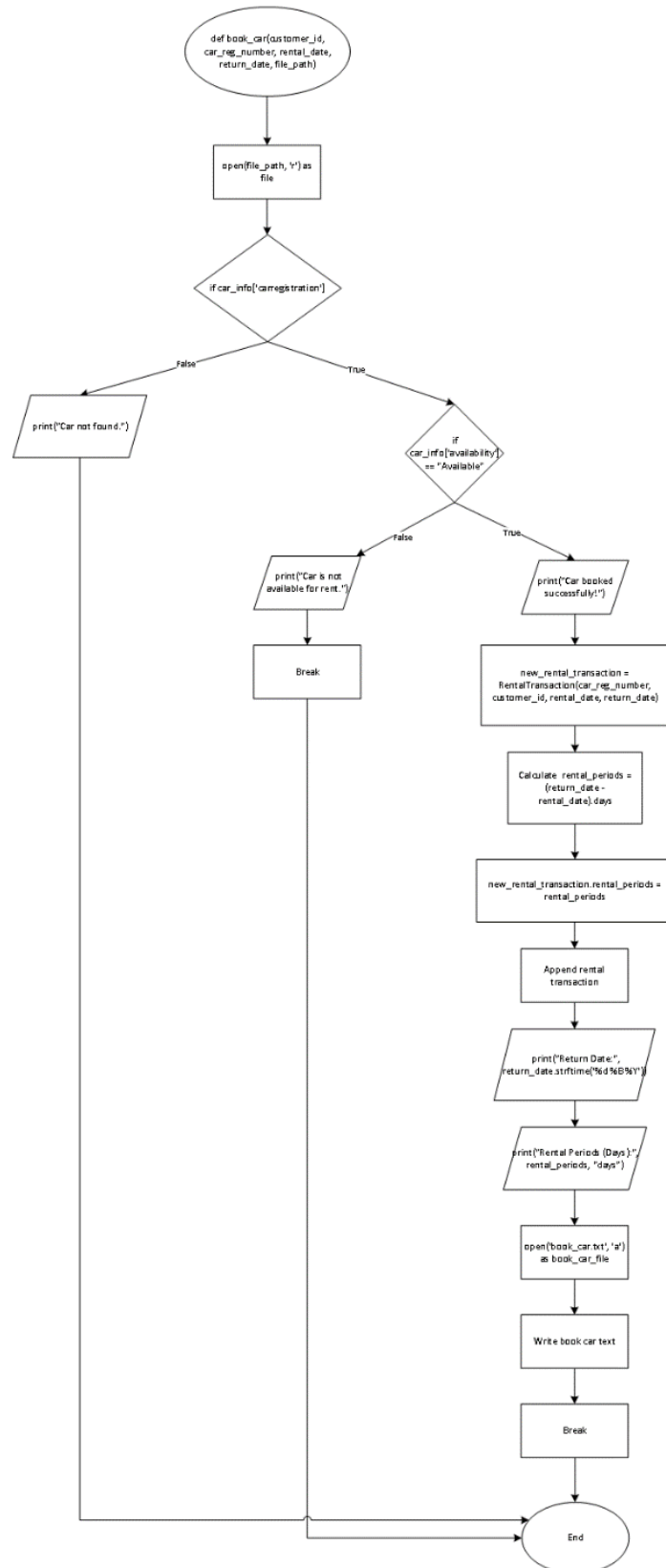
3.3.5 Accept Payment from Customer and Generate Receipt



```
define function generate_payment_and_receipt with parameters transaction and total_rental_amount
    generate payment_date as current date and time
    print confirmation message with accepted payment amount and customer ID
    create receipt_data with formatted receipt information including customer ID and total amount
    open "payment_and_receipt.txt" file for appending as file
    write receipt_data to file

set file_path to 'carinfo.txt'
initialize empty list customers to store Customer objects
initialize empty list cars to store Car objects
initialize empty list rental_transactions to store RentalTransaction objects
initialize current_user to None to store currently logged in user
initialize customer_id_counter to 0 to track customer IDs
initialize car_id_counter to 0 to track car IDs
```

3.3.6 Update “Reserved” when Customer Books Car



```
def book_car(customer_id, car_reg_number, rental_date, return_date, and
file_path):
    iterate over each line in file_path
        split line by ',' and create car_info dictionary
        if car registration number matches car_reg_number
            if car is available
                print booking success message
                create new RentalTransaction object with provided data
                calculate rental periods and update new_rental_transaction
                append new_rental_transaction to rental_transactions list
                write booking information to 'book_car.txt' file
            else
                print car not available message
            break if car is found
    print car not found message if loop completes without finding the car
```

3.3.7 Update “Rented” when Customer Rents Car



Define rent_car

Parameters: customer_id, car_reg_number, rental_date, return_date, cars, customers

Set customer_found as False

Loop over customers

 If customer.customer_id equals customer_id

 Set customer_found as True

 Break

If customer_found is False

 Print "Customer not found."

 Return False

Set car_found as False

Loop over cars

 If car.reg_number equals car_reg_number

 Set car_found as True

 If car.status is "Available" or "Reserved"

 Calculate rental_periods as (return_date - rental_date).days

 Set rental_price_per_day as car.rental_price

 Calculate total_rental as rental_periods * rental_price_per_day

 Format rental_date and return_date as strings

 Print "Car rented successfully!"

 Print "Rental Date:", rental_date_str

 Print "Return Date:", return_date_str

 Print "Rental Periods (Days):", rental_periods, "days"

 Print "Total Rental: RM{total_rental:.2f}"

 Write rental information to "car_rental.txt" file

 Return True

 Else

 Print "Car is not available for rent."

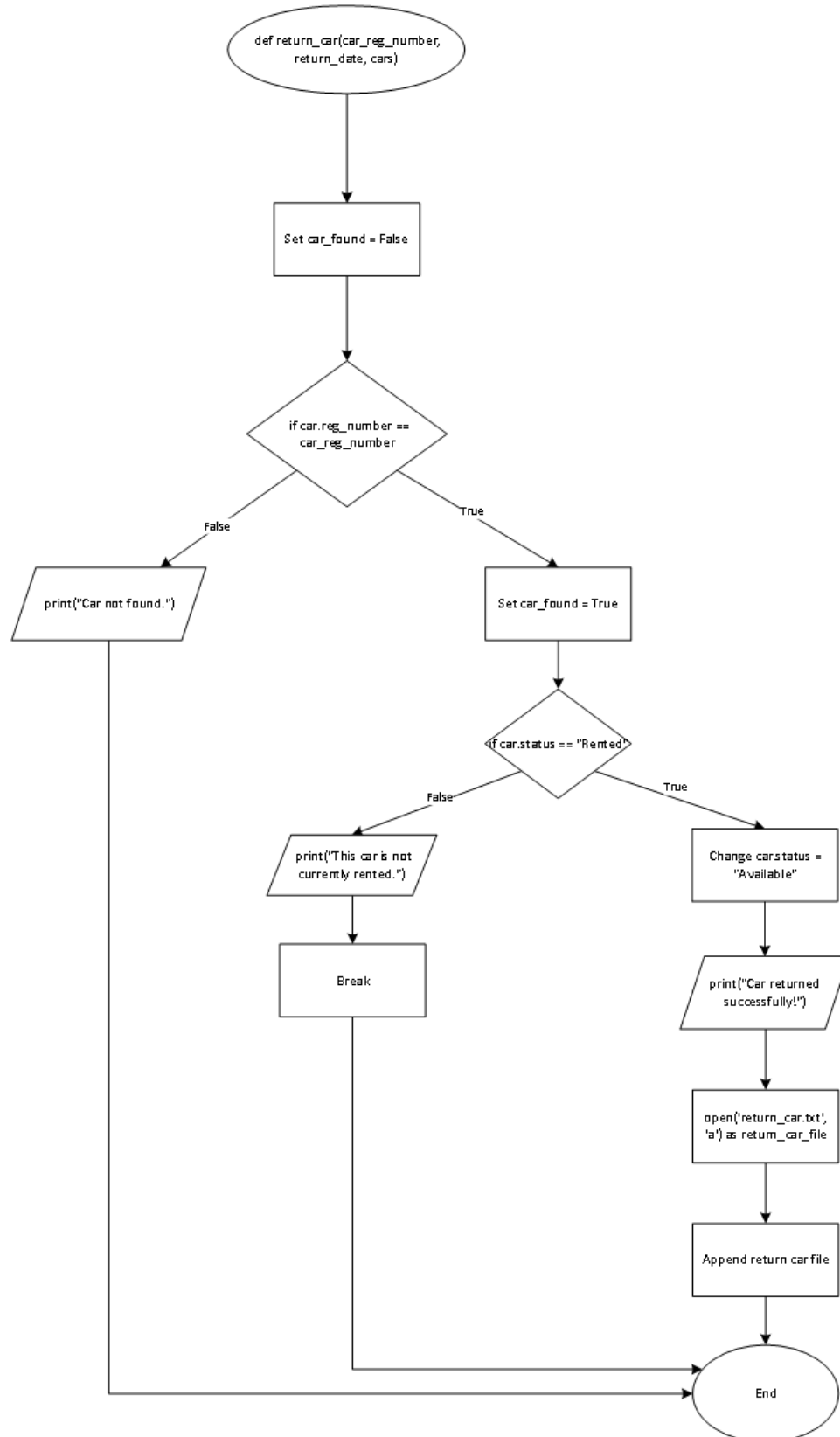
 Break

If car_found is False

 Print "Car not found."

Return False

3.3.8 Update “Available” when Customer Return Car



Define return_car

Parameters: car_reg_number, return_date, cars

Set car_found as False

If "car_rental.txt" exists

Open "car_rental.txt" in read mode

Read lines into car_rental_info

For each rental_record in car_rental_info

If car_reg_number is in rental_record

Set car_found as True

Print "Car returned successfully!"

Open "return_car.txt" in append mode as return_car_file

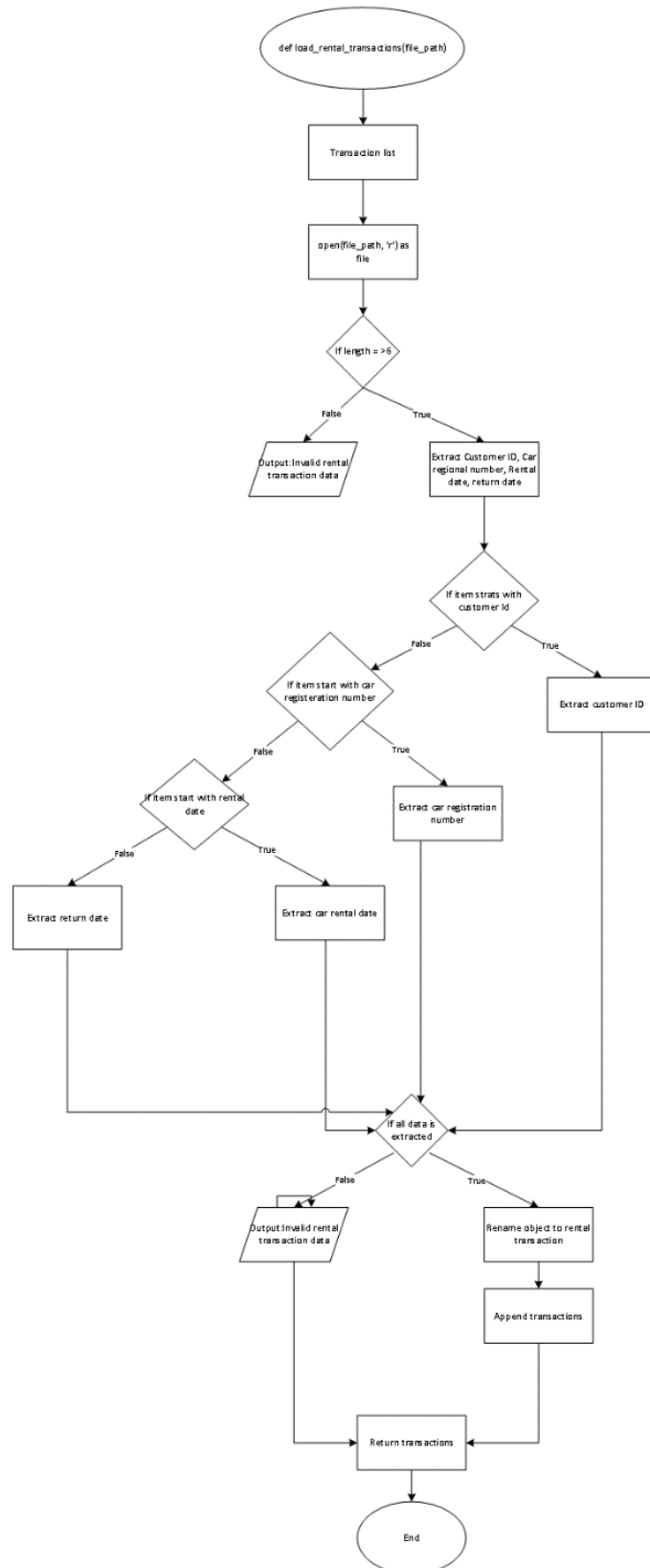
Write return information to return_car_file

Break

If car_found is False

Print "Car not found."

3.3.9 View List Rental Transactions



Define view_rental_transactions

Parameters: date

Create empty list rental_transactions_on_date

Open "car_rental.txt" in read mode as file

For each line in file

Split line into transaction_data by comma and space

Create empty dictionary transaction_info

For each item in transaction_data

Split item into key and value by colon and space

Add key-value pair to transaction_info dictionary

Parse rental_date from string to datetime

Parse return_date from string to datetime

If rental_date is equal to date

Create RentalTransaction object using data from file

Append rental_transaction to rental_transactions_on_date

If rental_transactions_on_date is empty

Print "No rental transactions found for the specified date."

Return

Print "Rental Transactions on", formatted date

For each transaction in rental_transactions_on_date

Print "Car Registration Number:", transaction.car_reg_number

Print "Customer ID:", transaction.customer_id

Print "Rental Date:", formatted transaction.rental_date

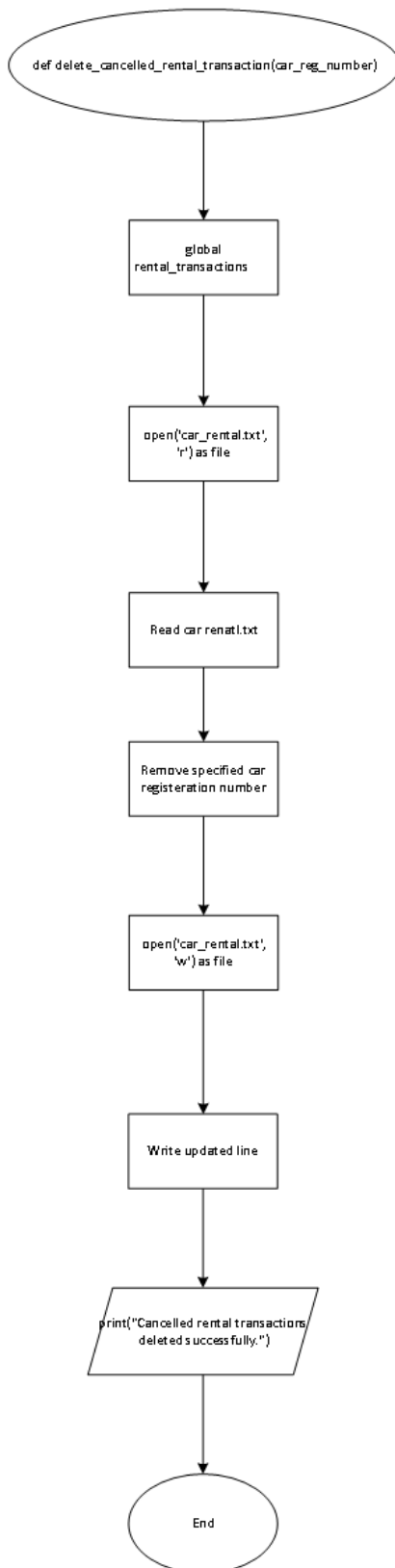
Print "Return Date:", formatted transaction.return_date

Print "Rental Periods (Days):", transaction.rental_periods

Print "Total Rental: RM{:.2f}".format(transaction.total_rental)

Print()

3.3.10 Delete the Cancelled Car Rental Transaction



Define delete_cancelled_rental_transaction

Parameters: car_reg_number

Set rental_transactions as global

Open "car_rental.txt" in read mode as file

Read lines into lines

Create updated_lines list without transactions containing car_reg_number

Open "car_rental.txt" in write mode as file

Write updated_lines to file

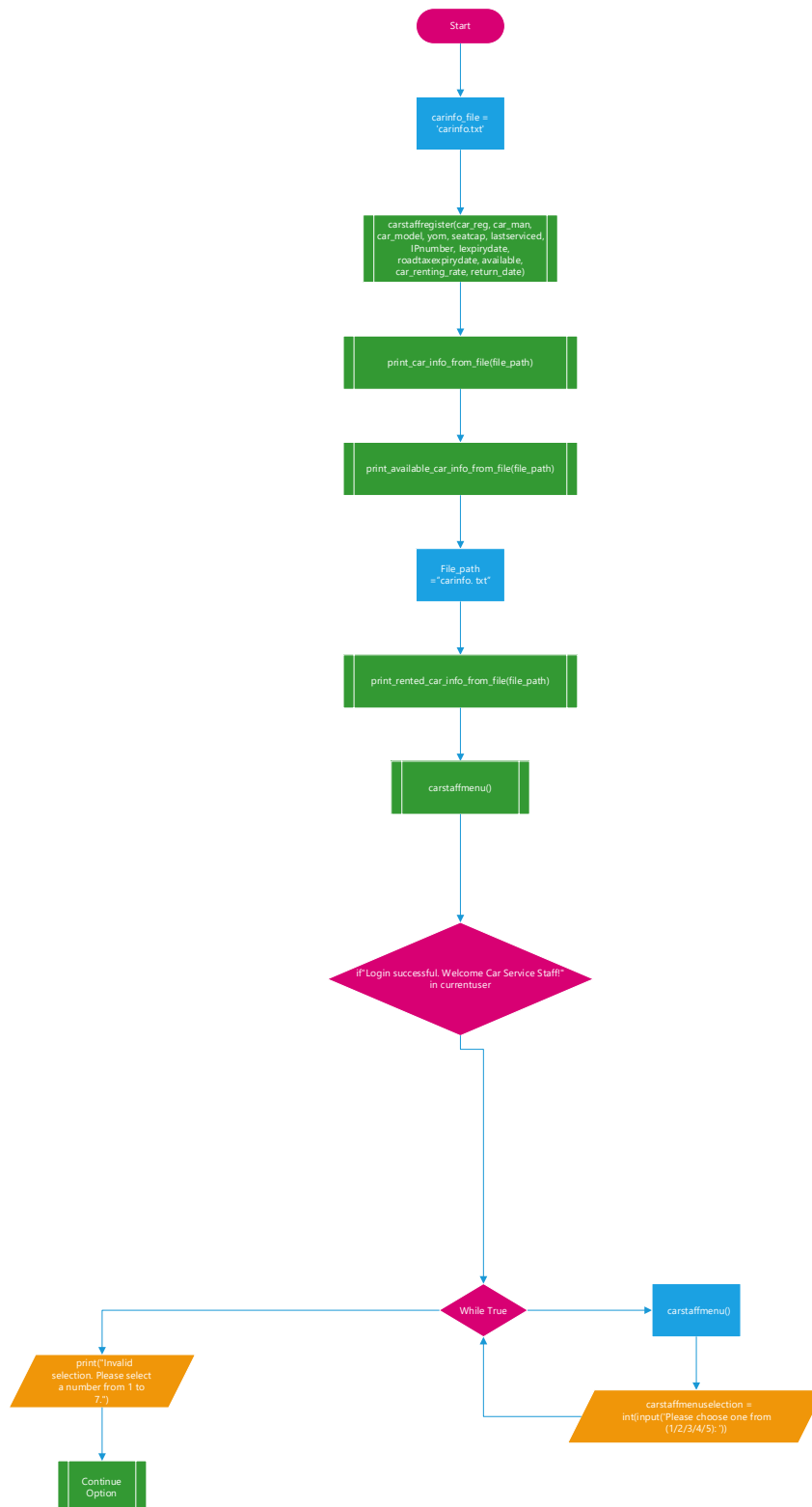
Set rental_transactions as filtered transactions without car_reg_number

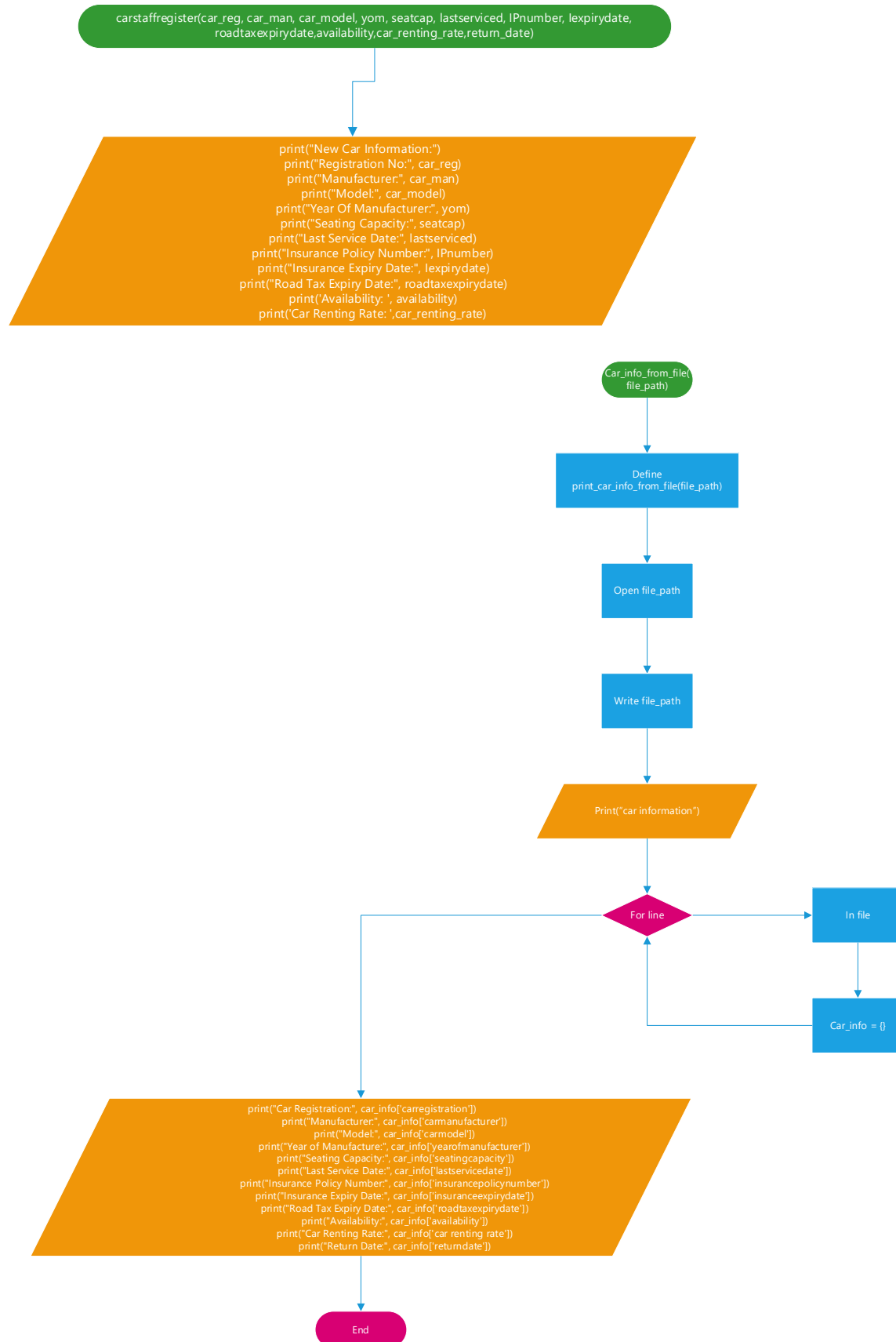
Print "Cancelled rental transactions deleted successfully."

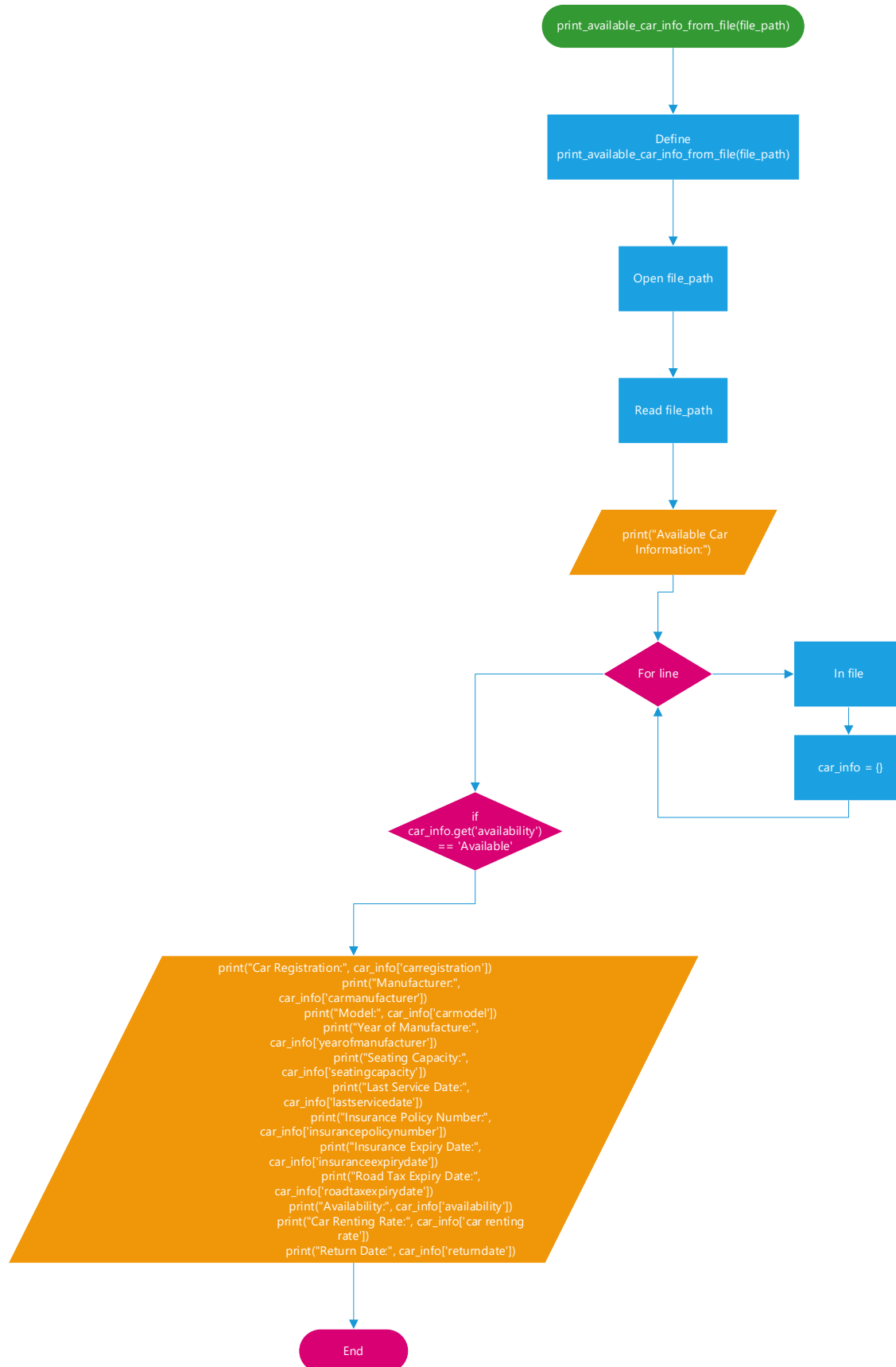
End

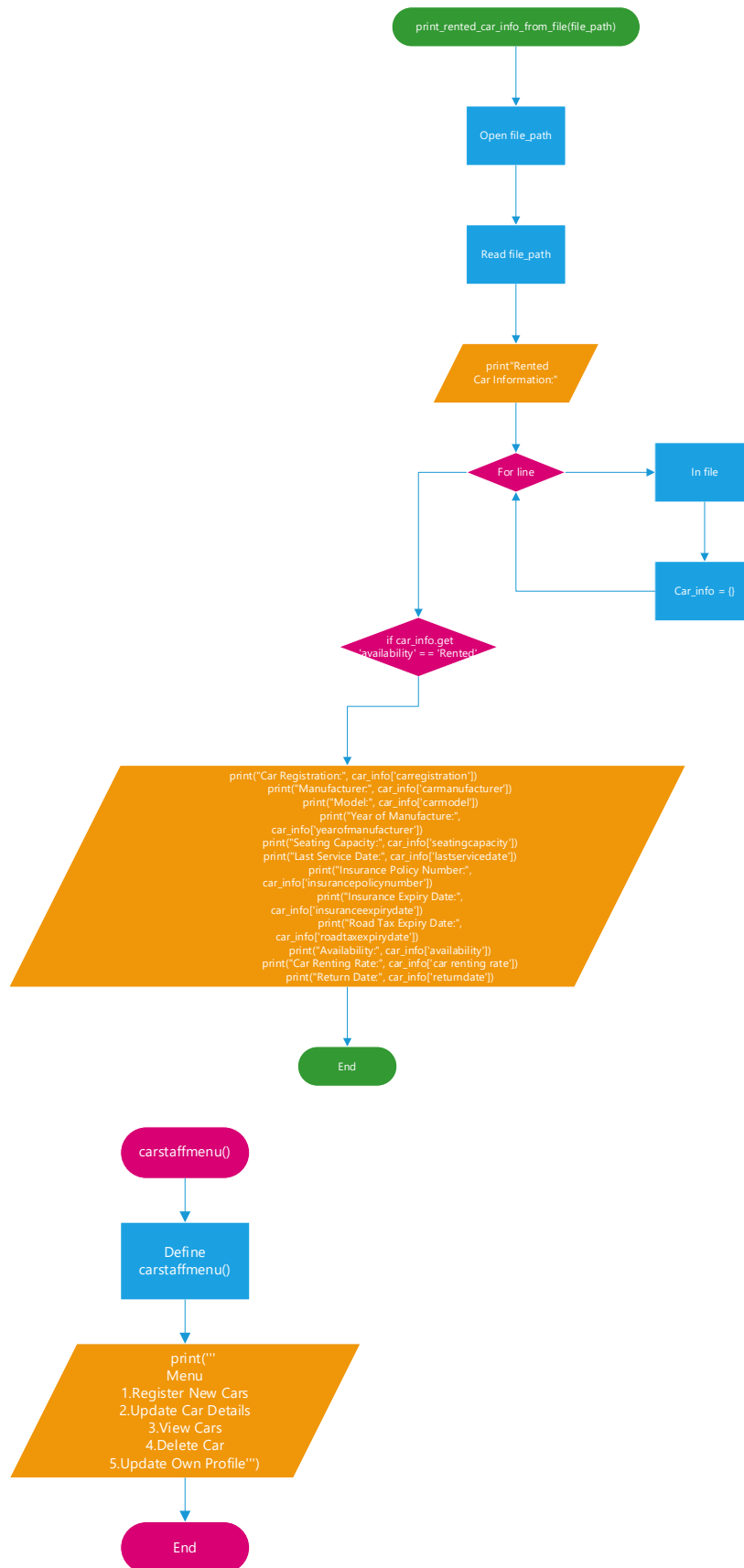
3.4 Car Service Staff

3.4.1 Main Menu









Start

Define `carstaffregister(car_reg, car_man, car_model, yom, seatcap, lastserved, IPnumber, Iexpirydate, roadtaxexpirydate, available, car_renting_rate, return_date)`:

- Create `new_car_info` dictionary with car details
- Open `carinfo_file` in append mode
- Format `new_car_info` into a string
- Write `formatted_item` to file with a newline character

Define `print_car_info_from_file(file_path)`:

- Open `file_path` in read mode
- Print "Car Information:"
- For each line in file:
 - Split line into `car_data` by comma and space
 - Create empty dictionary `car_info`
 - For each item in `car_data`:
 - Split item into key and value by colon and space
 - Add key-value pair to `car_info` dictionary
- Print "Car Registration:", `car_info['carregistration']`
- Print "Manufacturer:", `car_info['carmanufacturer']`
- Print "Model:", `car_info['carmodel']`
- Print "Year of Manufacture:", `car_info['yearofmanufacturer']`
- Print "Seating Capacity:", `car_info['seatingcapacity']`
- Print "Last Service Date:", `car_info['lastservicedate']`
- Print "Insurance Policy Number:", `car_info['insurancepolicynumber']`
- Print "Insurance Expiry Date:", `car_info['insuranceexpirydate']`
- Print "Road Tax Expiry Date:", `car_info['roadtaxexpirydate']`
- Print "Availability:", `car_info['availability']`
- Print "Car Renting Rate:", `car_info['car renting rate']`
- Print "Return Date:", `car_info['returndate']`

Define `print_available_car_info_from_file(file_path)`:

- Open `file_path` in read mode
- Print "Available Car Information:"
- For each line in file:
 - Split line into `car_data` by comma and space
 - Create empty dictionary `car_info`
 - For each item in `car_data`:

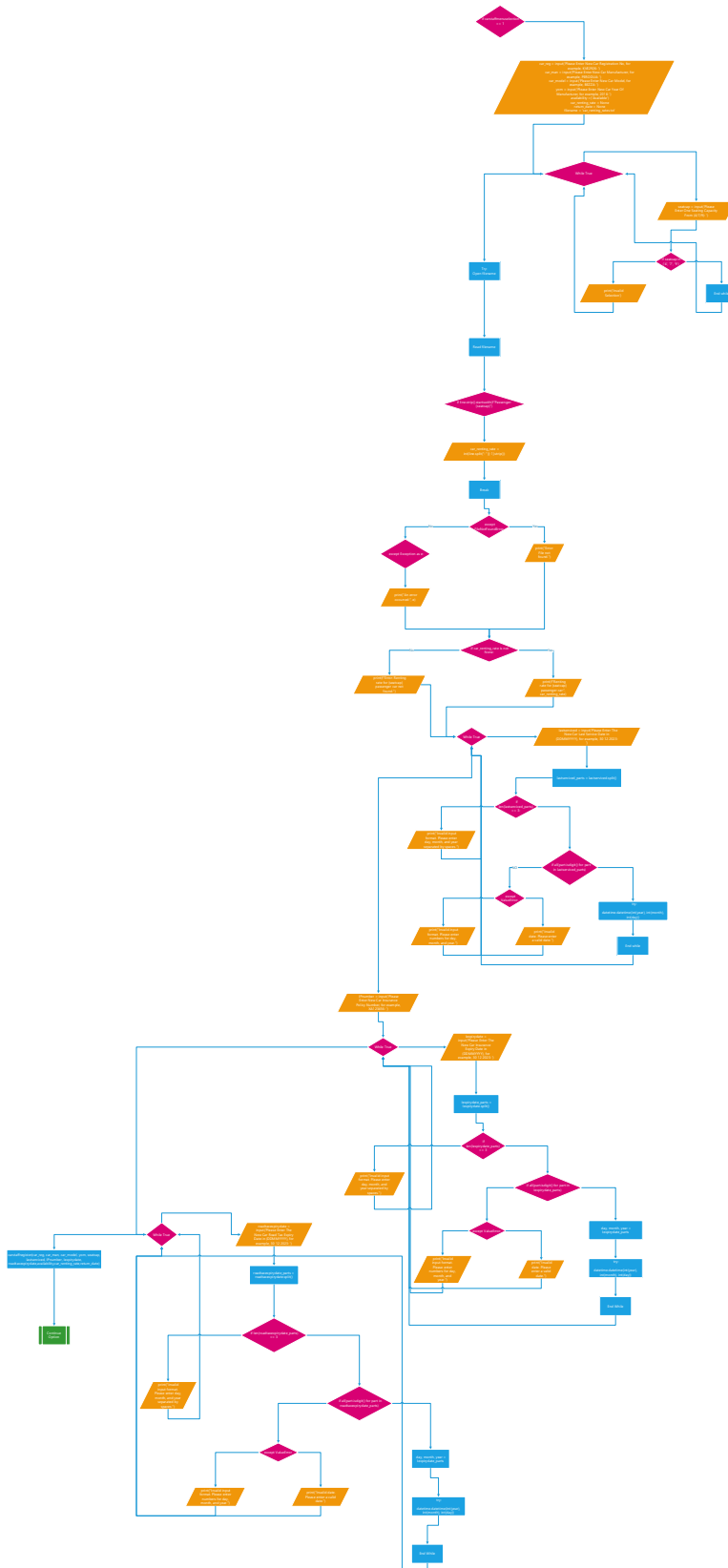
```
    Split item into key and value by colon and space
    Add key-value pair to car_info dictionary
If car_info['availability'] is 'Available':
    Print "Car Registration:", car_info['carregistration']
    Print "Manufacturer:", car_info['carmanufacturer']
    Print "Model:", car_info['carmodel']
    Print "Year of Manufacture:", car_info['yearofmanufacturer']
    Print "Seating Capacity:", car_info['seatingcapacity']
    Print "Last Service Date:", car_info['lastservicedate']
    Print "Insurance Policy Number:", car_info['insurancepolicynumber']
    Print "Insurance Expiry Date:", car_info['insuranceexpirydate']
    Print "Road Tax Expiry Date:", car_info['roadtaxexpirydate']
    Print "Availability:", car_info['availability']
    Print "Car Renting Rate:", car_info['car renting rate']
    Print "Return Date:", car_info['returndate']
```

Define print_rented_car_info_from_file(file_path):

```
    Open file_path in read mode
    Print "Rented Car Information:"
    For each line in file:
        Split line into car_data by comma and space
        Create empty dictionary car_info
        For each item in car_data:
            Split item into key and value by colon and space
            Add key-value pair to car_info dictionary
        If car_info['availability'] is 'Rented':
            Print "Car Registration:", car_info['carregistration']
            Print "Manufacturer:", car_info['carmanufacturer']
            Print "Model:", car_info['carmodel']
            Print "Year of Manufacture:", car_info['yearofmanufacturer']
            Print "Seating Capacity:", car_info['seatingcapacity']
            Print "Last Service Date:", car_info['lastservicedate']
            Print "Insurance Policy Number:", car_info['insurancepolicynumber']
            Print "Insurance Expiry Date:", car_info['insuranceexpirydate']
            Print "Road Tax Expiry Date:", car_info['roadtaxexpirydate']
            Print "Availability:", car_info['availability']
            Print "Car Renting Rate:", car_info['car renting rate']
```

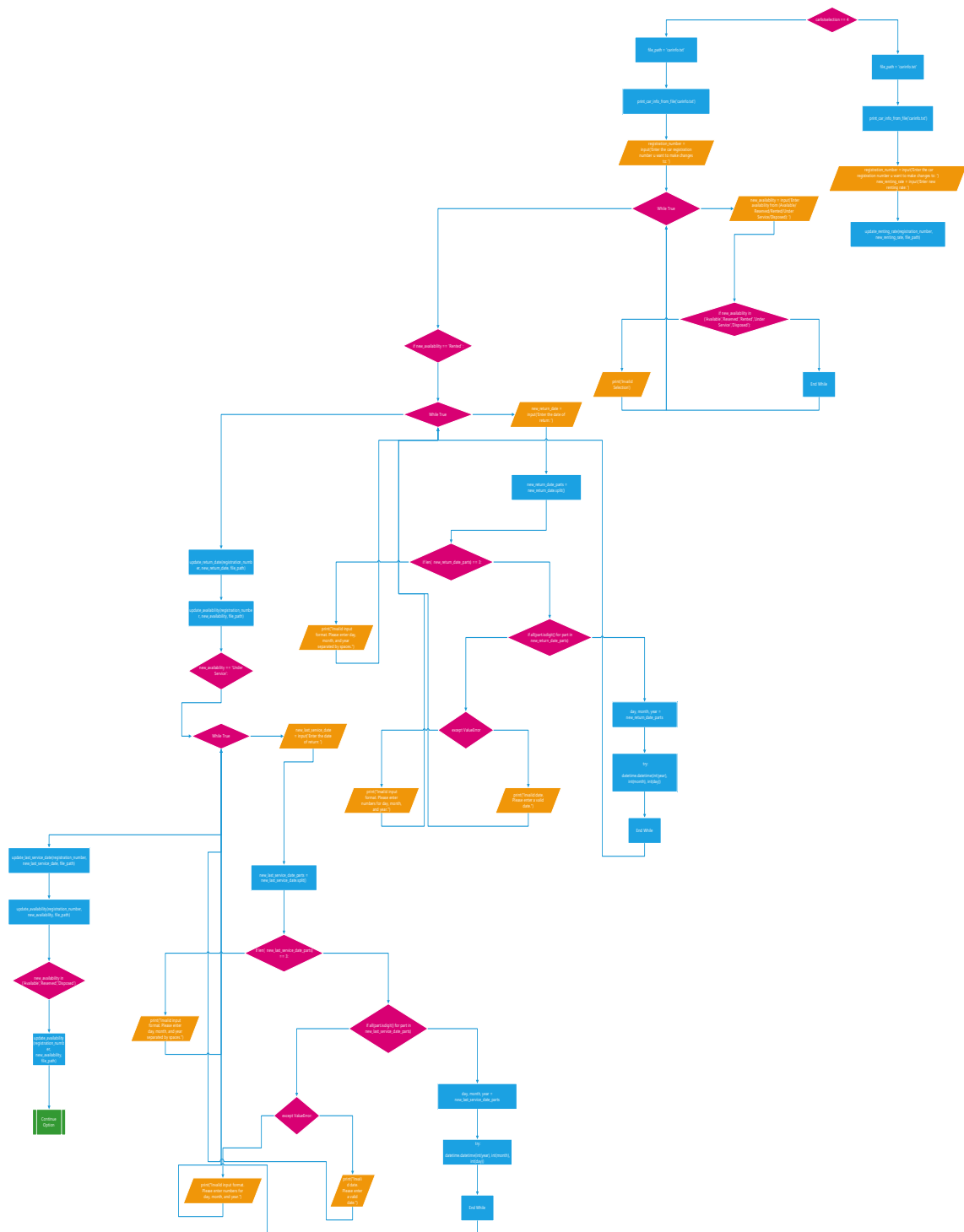
```
Print "Return Date:", car_info['returndate']  
define function: carstaffmenu  
display menu options:  
1. Register New Cars  
2. Update Car Details  
3. View Cars  
4. Delete Car  
5. Update Own Profile  
  
if "Login successful. Welcome Car Service Staff!" is in currentuser then  
loop indefinitely  
display carstaffmenu  
prompt user to choose an option (1/2/3/4/5)
```

3.4.2 Register new cars



```
if carstaffmenuselection is equal to 1 then
    prompt user for new car details
    prompt user for seating capacity until valid input is received
    open 'car_renting_rates.txt' for reading as file
    loop through each line in the file
        if line starts with "Passenger: {seatcap}" then
            set car_renting_rate to the integer value found in the line
            break from loop
    handle file not found exception
    handle other exceptions
    if car_renting_rate is not None then
        display renting rate for the selected seating capacity
    else
        display error message indicating renting rate not found
    prompt user for last service date until valid input is received
    prompt user for insurance policy number
    prompt user for insurance expiry date until valid input is received
    prompt user for road tax expiry date until valid input is received
    set availability to 'Available'
    set car_renting_rate to None
    set return_date to None
    call function carstaffregister with provided car details
    display new car information
```


[illegible]



define function: update_insurance_policy_number

parameters: registration_number, new_policy_number, file_path

initialize updated as False

read lines from file_path

loop over each line in lines

```
split line by ', ' into car_data
initialize car_info as empty dictionary
loop over each item in car_data
    split item by ': ' into key and value
    add key-value pair to car_info
if carregistration in car_info is equal to registration_number
    update insurancepolicynumber in car_info to new_policy_number
    set updated as True
    set updated_car_info to car_info
write updated car_info to file
if updated is True
    print "Insurance policy number updated successfully."
    print "Updated car information with appropriate labels:"
    loop over updated_car_info
        print key with value
else
    print "Car with the given registration number not found."
```

```
define function: update_insurance_expiry_date
parameters: registration_number, new_expiry_date, file_path
initialize updated_car_info as None
read lines from file_path
loop over each line in lines
    split line by ', ' into car_data
    initialize car_info as empty dictionary
    loop over each item in car_data
        split item by ': ' into key and value
        add key-value pair to car_info
    if carregistration in car_info is equal to registration_number
        update insuranceexpirydate in car_info to new_expiry_date
        set updated_car_info to car_info
    write updated car_info to file
if updated_car_info is not None
    print "Insurance expiry date updated successfully."
    print "Updated car information with appropriate labels:"
    loop over updated_car_info
        print key with value
```

```

else
    print "Car with the given registration number not found."

define function: update_roadtax_expiry_date
parameters: registration_number, roadtax_expiry_date, file_path
initialize updated_car_info as None
read lines from file_path
loop over each line in lines
    split line by ', ' into car_data
    initialize car_info as empty dictionary
    loop over each item in car_data
        split item by ': ' into key and value
        add key-value pair to car_info
    if carregistration in car_info is equal to registration_number
        update roadtaxexpirydate in car_info to roadtax_expiry_date
        set updated_car_info to car_info
    write updated car_info to file
if updated_car_info is not None
    print "Road Tax expiry date updated successfully."
    print "Updated car information with appropriate labels:"
    loop over updated_car_info
        print key with value
else
    print "Car with the given registration number not found."

define update_renting_rate(registration_number, new_renting_rate, file_path)
updated_car_info = None
open file_path for reading as file
    lines = read lines from file

open file_path for writing as file
for each line in lines do
    car_data = split line by stripping and splitting by ', '
    car_info = {}
    for each item in car_data do
        key, value = split item by ': '
        car_info[key stripped] = value stripped
    if car_info's carregistration is equal to registration_number then

```

```
        car_info['car renting rate'] = new_renting_rate
        updated_car_info = car_info
        write ', '.join(key: value for each key, value in car_info) to file with newline

    if updated_car_info exists then
        print "Renting rate updated successfully."
        print newline
        print "Updated car information:"
        for each key, value in updated_car_info do
            print key: value
    else
        print "Car with the given registration number not found."

def update_availability(registration_number, new_availability, file_path)
    updated_car_info = None
    open file_path for reading as file
        lines = read lines from file

    open file_path for writing as file
        for each line in lines do
            car_data = split line by stripping and splitting by ', '
            car_info = {}
            for each item in car_data do
                key, value = split item by ': '
                car_info[key stripped] = value stripped
            if car_info['carregistration'] is equal to registration_number then
                car_info['availability'] = new_availability
                updated_car_info = car_info
            write ', '.join(key: value for each key, value in car_info) to file with newline

    if updated_car_info exists then
        print "Availability updated successfully."
        print newline
        print "Updated car information:"
        for each key, value in updated_car_info do
            print key: value
    else
```

```

    print "Car with the given registration number not found."

define update_return_date(registration_number, new_return_date, file_path)
    open file_path for reading as file
        lines = read lines from file

    open file_path for writing as file
        for each line in lines do
            car_data = split line by stripping and splitting by ', '
            car_info = {}
            for each item in car_data do
                key, value = split item by ':'
                car_info[key stripped] = value stripped
            if car_info's carregistration is equal to registration_number then
                car_info's 'returndate' = new_return_date
            write ', '.join(key: value for each key, value in car_info) to file with newline

define update_last_service_date(registration_number, new_last_service_date, file_path)
    open file_path for reading as file
        lines = read lines from file

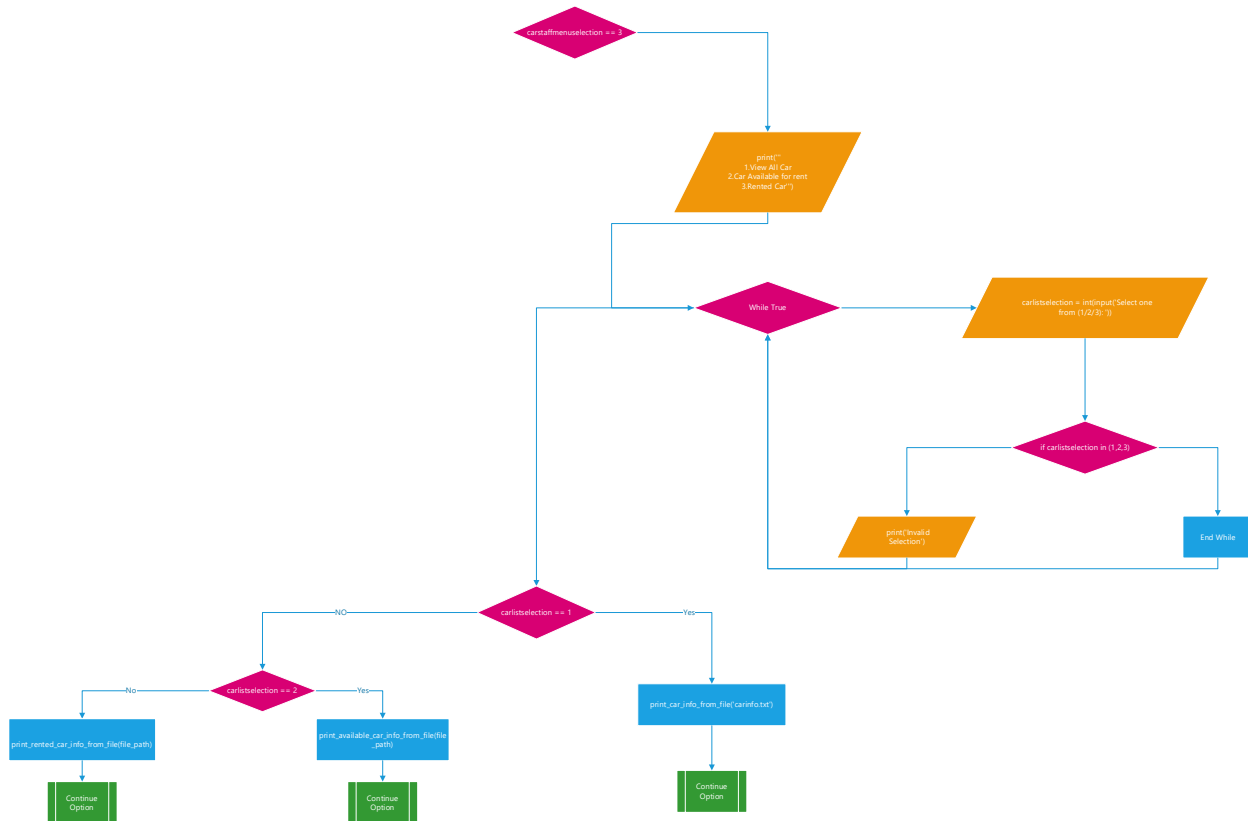
    open file_path for writing as file
        for each line in lines do
            car_data = split line by stripping and splitting by ', '
            car_info = {}
            for each item in car_data do
                key, value = split item by ':'
                car_info[key stripped] = value stripped
            if car_info's carregistration is equal to registration_number then
                car_info's 'lastservicedate' = new_last_service_date
            write ', '.join(key: value for each key, value in car_info) to file with newline
    else if carstaffmenuselection is equal to 2 then
        display update menu
        prompt user to select an option (1/2/3/4/5)
        loop indefinitely
            prompt user to select an option (1/2/3/4/5)
            if user input is valid then

```

```
        break from loop
    else
        display error message indicating invalid selection
if selected option is 1 then
    prompt user to enter car registration number
    prompt user to enter new insurance policy number
    call function update_insurance_policy_number with provided inputs
else if selected option is 2 then
    prompt user to enter car registration number
    loop indefinitely
        prompt user to enter new insurance expiry date until valid input is received
        if input is valid then
            break from loop
        else
            display error message indicating invalid date format
    call function update_insurance_expiry_date with provided inputs
else if selected option is 3 then
    prompt user to enter car registration number
    loop indefinitely
        prompt user to enter new road tax expiry date until valid input is received
        if input is valid then
            break from loop
        else
            display error message indicating invalid date format
    call function update_roadtax_expiry_date with provided inputs
else if selected option is 4 then
    prompt user to enter car registration number
    prompt user to enter new renting rate
    call function update_renting_rate with provided inputs
else if selected option is 5 then
    prompt user to enter car registration number
    loop indefinitely
        prompt user to enter new availability until valid input is received
        if input is valid then
            break from loop
        else
            display error message indicating invalid selection
```

```
if new availability is 'Rented' then
    loop indefinitely
        prompt user to enter return date until valid input is received
        if input is valid then
            break from loop
        else
            display error message indicating invalid date format
    call function update_return_date with provided inputs
else if new availability is 'Under Service' then
    loop indefinitely
        prompt user to enter last service date until valid input is received
        if input is valid then
            break from loop
        else
            display error message indicating invalid date format
    call function update_last_service_date with provided inputs
call function update_availability with provided inputs
```


3.4.4 View Lists of Cars



else if carstaffmenuselection is equal to 3 then

display car view options

prompt user to select an option (1/2/3)

loop indefinitely

prompt user to select an option (1/2/3)

if user input is valid then

break from loop

else

display error message indicating invalid selection

if selected option is 1 then

display all car information

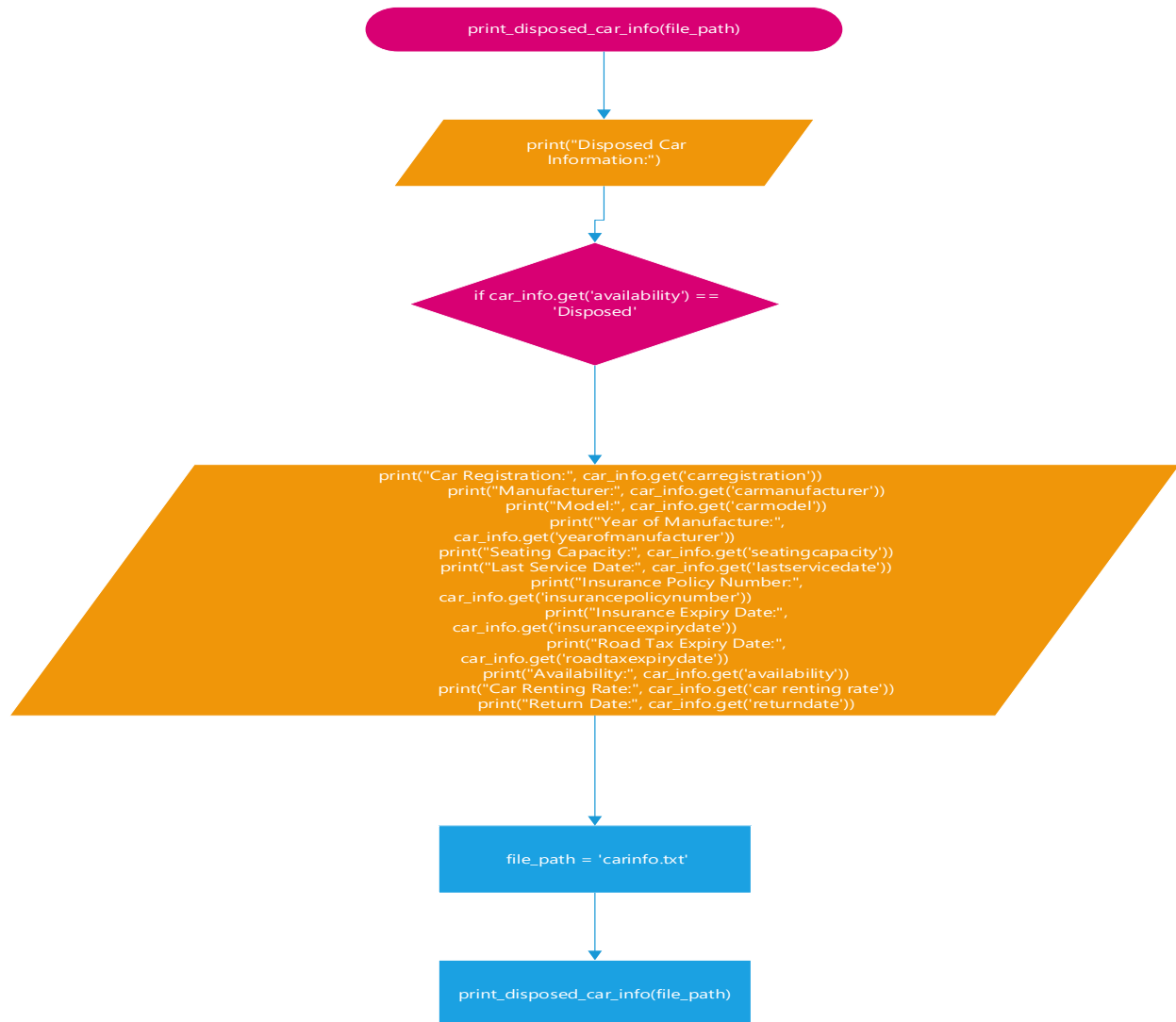
else if selected option is 2 then

display available car information

else if selected option is 3 then

display rented car information

3.4.5 Delete Car where availability is “Disposed”



elif carstaffmenuselection is equal to 4 then

define function `print_disposed_car_info` with parameter `file_path`

print "Disposed Car Information:"

open `file_path` for reading as file

loop through each line in the file

split the line into car data

initialize an empty dictionary `car_info`

loop through each item in car data

split the item into key and value

add key-value pair to `car_info` dictionary

if car availability is 'Disposed' then

print car registration number, manufacturer, model, year of manufacture, seating capacity,
last service date, insurance policy number, insurance expiry date, road tax expiry date,
availability, car renting rate, and return date

call function print_disposed_car_info with argument 'carinfo.txt'

define function delete_disposed_car_by_registration with parameter file_path

prompt user to enter the registration number of the car to delete

open file_path for reading as file

read all lines from the file

open file_path for writing as file

initialize car_deleted to False

loop through each line in the lines

split the line into car data

initialize an empty dictionary car_info

loop through each item in car data

split the item into key and value

add key-value pair to car_info dictionary

if car registration number matches input registration number and car availability is 'Disposed' then

print "Car with registration number '{registration_number}' deleted successfully."

set car_deleted to True

else

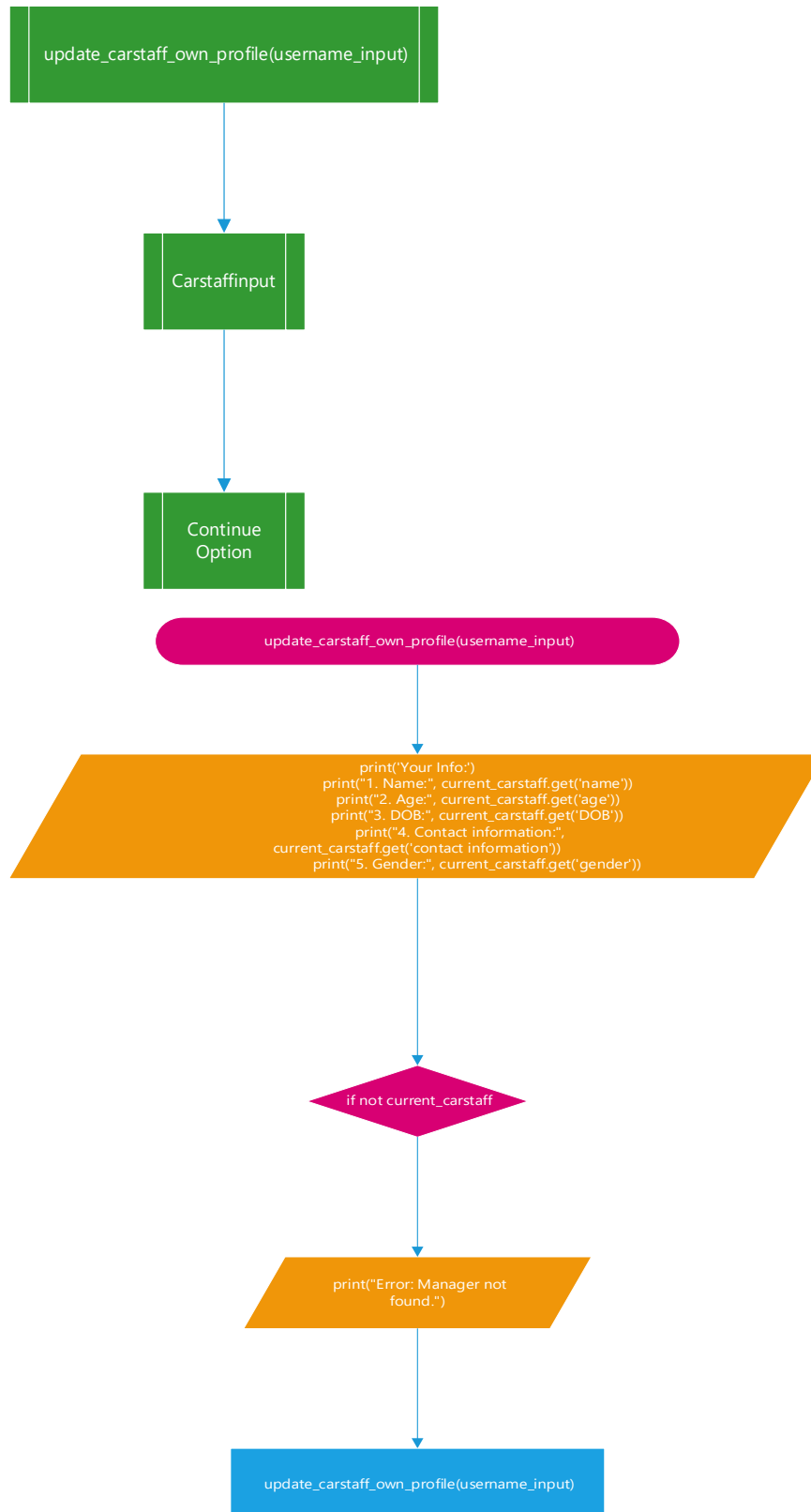
write the line back to the file

if car_deleted is False then

print "Car not found or not disposed."

call function delete_disposed_car_by_registration with argument 'carinfo.txt'

3.4.6 Update Profile



```
prompt user to select an option (1/2/3/4/5)
loop indefinitely
    prompt user to select an option (1/2/3/4/5)
    if user input is valid then
        break from loop
    else
        print "Invalid Selection"

define function update_carstaff_info with parameters field, new_value, and username
    set file_path to 'car_service_staff.txt'
    initialize updated_data list to store updated data
    initialize updated to False
    open file_path for reading as file
        loop through each line in the file
            if line contains username then
                split the line into data
                initialize an empty dictionary carstaff_info
                loop through each info in data
                    split info into key and value
                    if key matches field then
                        set carstaff_info[key] to new_value
                        set updated to True
                    else
                        add key-value pair to carstaff_info dictionary
                append carstaff_info to updated_data list
            else
                append line to updated_data list
    open file_path for writing as file
        loop through each item in updated_data
            if item is a dictionary then
                write key-value pairs as string to the file
                break from loop
            else
                write the item to the file
    if updated is True then
        print "Field updated successfully."
    else
```

```
    print "Error: Username not found."

    if carstaffinfoselection is equal to 1 then
        prompt user to enter new name
            call function update_carstaff_info with field as 'name', new_value as newinfofname, and
            username as username_input
    if carstaffinfoselection is equal to 2 then
        prompt user to enter new age
            call function update_carstaff_info with field as 'age', new_value as newinfoage, and username
            as username_input
    if carstaffinfoselection is equal to 3 then
        prompt user to enter new DOB
            call function update_carstaff_info with field as 'DOB', new_value as newinfoDOB, and
            username as username_input
    if carstaffinfoselection is equal to 4 then
        prompt user to enter new contact information
            call function update_carstaff_info with field as 'contact information', new_value as newinfoCI,
            and username as username_input
    if carstaffinfoselection is equal to 5 then
        prompt user to enter new gender
            call function update_carstaff_info with field as 'gender', new_value as newinfogender, and
            username as username_input

    pass

else
    print "Invalid selection. Please select a number from 1 to 7."

loop indefinitely
    prompt user to continue or exit
    if user inputs 'yes' then
        continue loop
    else
        break from loop

End
```

4.0 Explanation of Programming Concepts

1. **File I/O:** The system uses file I/O to read and write data from/to text files. For example, the `delete_cancelled_rental_transaction` function demonstrates File I/O by removing a specific car's rental transaction from the `car_rental.txt` file and updating the global `rental_transactions` list. It reads all lines from the file, filters out the transaction with the given car registration number, and writes the updated lines back to the file. The function also updates the global list by excluding the deleted transaction. This ensures both the file and the list reflect the removal accurately, maintaining data consistency. Finally, it confirms the deletion with a printed message.

```
def delete_cancelled_rental_transaction(car_reg_number):
    # This function deletes a rental transaction associated with a specific car registration number.
    # It removes the transaction from the 'car_rental.txt' file and updates the global list
    # of rental_transactions accordingly.
    global rental_transactions

    # Read all rental transactions from the car_rental.txt file
    with open('car_rental.txt', 'r') as file:
        lines = file.readlines()

    # Filter out the transactions with the specified car registration number
    updated_lines = [line for line in lines if car_reg_number not in line]

    # Write the filtered transactions back to the car_rental.txt file
    with open('car_rental.txt', 'w') as file:
        file.writelines(updated_lines)

    # Update the rental_transactions list by filtering out the deleted transactions
    rental_transactions = [transaction for transaction in rental_transactions if transaction.car_reg_number != car_reg_number]

    print("Cancelled rental transactions deleted successfully.")
```

2. Data structures: The system uses data structures such as lists and dictionaries to store and manipulate data. For example, `updated_data_list` is used to store the updated manager information. Each element in the list represents a line from the `manager.txt` file, which can either be a string (an unmodified line) or a dictionary (an updated manager's information). Each manager's information is stored in a dictionary. This dictionary contains key-value pairs where each key represents a field (e.g., `username`, `email`) and the corresponding value is the field's data.

```
def update_manager_info(field, new_value, username):
    # Define the file path
    file_path = 'manager.txt'

    # Read data from the file
    updated_data = []
    updated = False
    with open(file_path, 'r') as file:
        for line in file:
            if f"username: {username}" in line:
                data = line.strip().split(' ')
                manager_info = {}
                for info in data:
                    key, value = info.split(': ')
                    if key.strip() == field:
                        # Update the specified field
                        manager_info[key.strip()] = new_value
                        updated = True
                    else:
                        manager_info[key.strip()] = value.strip()
                updated_data.append(manager_info) # Append manager_info after processing all fields
            else:
                updated_data.append(line.strip())

    # Write the updated data back to the file
    with open(file_path, 'w') as file:
        for item in updated_data:
            if isinstance(item, dict):
                file.write(', '.join([f"{key}: {value}" for key, value in item.items()]) + '\n')
                break
            else:
                file.write(item + '\n')
```


3. Functions: The system uses functions to capture specific functionality and make the code more modular and reusable. For example, the `view_customers` function displays customer details in a formatted table. It uses the `load_customers` function to load customer details from a specified file. If no customers are registered, it prints a message and returns. If customers are registered, it prints their details in a formatted table using a for loop.

```
def view_customers(file_path):
    # Load customer details from the file
    customers = load_customers(file_path)

    # Check if there are any customers registered
    if not customers:
        print("No customers registered yet!")
        return

    # Print the details of each customer in a formatted table
    print("{:<20} {:<30} {:<20} {:<30} {:<20} {:<20}".format("Customer ID", "Name", "NRIC/Passport", "License", "Address", "Phone", "Registration Date"))
    for customer in customers:
        print("{:<20} {:<30} {:<20} {:<30} {:<20} {:<20}".format(customer.customer_id, customer.name, customer.nric or customer.passport, customer.license, customer.address, customer.phone, customer.registration_date))
```

```
elif choice_staff == '3':
    view_customers("customers.txt")
```

4. Classes: The system uses classes to define custom data types and their behaviour. (HackerEarth, 2024) For example, the `RentalTransaction` class is used to represent a rental transaction and its attributes.

```
class RentalTransaction:
    def __init__(self, car_registration_number, customer_id, rental_date, return_date, total_rental):
        self.car_registration_number = car_registration_number
        self.customer_id = customer_id
        self.rental_date = rental_date
        self.return_date = return_date
        self.total_rental = total_rental
```

5. Error handling: The system uses error handling to manage exceptions and provide meaningful error messages to the user. For example, the login function checks if the provided username and password are valid and prints an error message if they are not.

```
attempts += 1 # Increment the attempts counter
print(f"Error: Invalid username or password. You have {3 - attempts} attempt(s) remaining.")
if attempts < 3:
    username_input = input("Username: ") # Prompt for username again
    password_input = input("Password: ") # Prompt for password again

return "Maximum login attempts reached. Please try again later."
```

6. **User input:** The system uses user input to get data from the user and make the system interactive. For example, the login function prompts the user to enter their username and password.

```
# Get username and password from user input
username_input = input("Username: ")
password_input = input("Password: ")

# Call the login function with the provided arguments
currentuser = login(username_input, password_input)
```

7. **String manipulation:** The system uses string manipulation to parse and format data. (Thevapalan, 2023) For example, the view_monthly_revenue function uses string manipulation to parse the rental dates from the car_rental.txt file.

```
with open('car_rental.txt', 'r') as file:
    for line in file:
        transaction_data = line.strip().split(',')
        transaction_info = {}
        for item in transaction_data:
            key, value = item.split(': ')
            transaction_info[key.strip()] = value.strip()

        # Parse dates from string to datetime
        rental_date = datetime.datetime.strptime(transaction_info['Rental Date'], "%d %B %Y").date()
        return_date = datetime.datetime.strptime(transaction_info['Return Date'], "%d %B %Y").date()
```

8. **Control structures:** The system uses control structures such as if-else statements and loops to control the flow of the program. For example, the update_car_status function uses an if-else statement to use a for loop to iterate over the list of cars, checking each car's registration number against the specified reg_number. Within the loop, an if statement determines if the current car's registration number matches the given reg_number. If a match is found, the car's status is updated to new_status, and a confirmation message is printed. If the loop completes without finding a matching car, the else statement executes, printing a message indicating that the car was not found.

```
def update_car_status(reg_number, new_status, cars):
    # This function updates the status of a car with a specified registration number.
    # It modifies the status of the car object in the list of cars and updates the status
    # in the car information file.
    for car in cars:
        if car.reg_number == reg_number:
            car.status = new_status
            print(f"Status of car with registration number {reg_number} updated to {new_status}.")
            break
    else:
        print(f"Car with registration number {reg_number} not found.")
```

9. Data validation: The system uses data validation to ensure that the user-provided data is valid and meets certain criteria. For example, this part of code uses a while True loop to repeatedly prompt the user to enter a phone number until a valid format is provided. Inside the loop, the input is stripped of leading and trailing whitespace and then checked to ensure it contains only digits and hyphens. If the input is valid, the loop breaks, allowing the program to proceed. If the input is invalid, an error message is displayed, and the user is prompted to try again.

```
while True:
    phone = input("Enter customer phone number (should contain only numbers and hyphens): ").strip()
    if all(c.isdigit() or c == '-' for c in phone):
        break
    else:
        print("Invalid phone number format. Phone number should contain only numbers and hyphens (-). Please try again.")
```

10. Modularity: The system uses modularity to break down the code into smaller, more manageable pieces and make it easier to maintain and test. (GeeksforGeeks, 2024) For example, the menu_1 function demonstrates modularity by encapsulating menu options and their corresponding actions for Customer Service Staff I within a single function. Each menu option, such as registering customers or updating customer details, is handled by a separate function called within the menu_1 function. This approach separates concerns, ensuring that each function focuses on a specific task, promoting clarity and maintainability.

```
def menu_1():
    # This function displays the Customer Service Staff I menu options and handles user input accordingly.
    # It allows staff to register customers, update customer details, view customer information,
    # update their own profile, and delete customer records.
    global customers # Ensure we're using the global 'customers' variable
    # Load existing customers
    customers = load_customers("customers.txt")
    profile_file_path = "customer_staff_profile.txt"
    while True:
        print("\nCustomer Service Staff I Menu")
        print("1. Register Customer")
        print("2. Update Customer Details")
        print("3. View Customers")
        print("4. Update Profile")
        print("5. Delete Customer")
        print("6. Back to Main Menu")
        choice_staff = input("\nEnter your choice (1-6): ")
        if choice_staff not in ('1', '2', '3', '4', '5', '6'):
            print("Invalid choice. Please try again.")
            continue
        if choice_staff == '1':
            register_customer()
        elif choice_staff == '2':
            customer_id = input("Enter customer ID to update: ")
            update_customer_details(customer_id)
        elif choice_staff == '3':
            view_customers("customers.txt")
        elif choice_staff == '4':
            update_profile(profile_file_path)
        elif choice_staff == '5':
            customer_id = input("Enter customer ID: ")
            delete_customer(customer_id)
        elif choice_staff == '6':
            break # Exit Customer Service Staff I Menu
```

5.0 Additional Features Source Code with Explanation

Countdown Mechanism:

```
while attempts < 3: # Continue the loop until three attempts are reached
    username = username_input # Assign the input username to a local variable
    password = password_input # Assign the input password to a local variable

    # Helper function to check login in a given file
    def check_login(file_name):
        try:
            with open(file_name, 'r') as file:
                for line_num, line in enumerate(file, start=1):
                    if f"username: {username}" in line:
                        data = line.strip().split(', ')
                        current_user = {}
                        for info in data:
                            try:
                                key, value = info.split(': ')
                                current_user[key.strip()] = value.strip()
                            except ValueError:
                                print(f"Skipping line {line_num} with invalid data format in file '{file_name}'")
                                continue
                        if current_user.get('password') == password:
                            return f"Login successful. Welcome {current_user.get('role')}!"
        except FileNotFoundError:
            print(f"Error: The file '{file_name}' was not found.")
        except ValueError as ve:
            print(f"Error processing file '{file_name}': {ve}")
        return None

    # Check for login in different roles
    result = check_login('manager.txt') or check_login('customer_staff.txt') or check_login('car_service_staff.txt')

    if result:
        return result

    attempts += 1 # Increment the attempts counter
    print(f"Error: Invalid username or password. You have {3 - attempts} attempt(s) remaining.")
    if attempts < 3:
        username_input = input("Username: ") # Prompt for username again
        password_input = input("Password: ") # Prompt for password again

    return "Maximum login attempts reached. Please try again later."
```

The login function implements a countdown mechanism to limit login attempts to three. It initializes an attempts counter and uses a while loop to allow up to three tries. The function checks the username and password against records in manager.txt, customer_staff.txt, and car_service_staff.txt by reading each file and verifying the credentials. If the credentials match, it returns a success message. If not, it increments the attempts counter, informs the user of the remaining attempts, and prompts for the username and password again. After three failed attempts, the function returns a message indicating the maximum login attempts have been reached. This mechanism helps prevent unauthorized access by limiting repeated login attempts.

Auto Generate Customer ID:

```
def generate_customer_id():
    # This function generates a unique customer ID by reading the existing customer IDs from the file,
    # finding the maximum ID, incrementing it by 1, and formatting it with leading zeros.
    max_customer_id = 0
    if os.path.exists("customers.txt"):
        with open("customers.txt", "r") as file:
            for line in file:
                fields = line.strip().split("|")
                if fields:
                    customer_id_str = fields[0].split(":")[1].strip()[1:] # Extract customer ID string
                    try:
                        customer_id = int(customer_id_str)
                        max_customer_id = max(max_customer_id, customer_id)
                    except ValueError:
                        pass # Silently handle the error without printing

    next_customer_id = max_customer_id + 1
    next_customer_id = max(next_customer_id, 1) # Ensure the next ID is at least 1
    next_customer_id_str = str(next_customer_id).zfill(6) # Format the ID with leading zeros
    return f"C{next_customer_id_str}"
```

The `generate_customer_id` function ensures the creation of a unique customer ID by processing existing IDs from the `customers.txt` file. It begins by initializing `max_customer_id` to 0 and checking if the file exists. If it does, the function reads the file line by line, splitting each line into fields to extract the customer ID. It isolates the numeric part of the ID, converts it to an integer, and updates `max_customer_id` if the extracted ID is larger. Any conversion errors are silently handled. After processing all lines, the function increments `max_customer_id` by 1, ensuring the new ID is at least 1. The next customer ID is then formatted to six digits with leading zeros and returned with a "C" prefix, guaranteeing a unique, sequential, and consistently formatted ID.

Current Date:

```
registration_date = datetime.datetime.now().strftime("%d %B %Y")

# Writing to customers.txt
with open("customers.txt", "a") as file:
    if id_type == "a":
        file.write(f"Customer ID: {customer_id} | Name: {name} | NRIC: {nric} | License: {license} | Address: {address} | Phone: {phone} | Registration Date: {registration_date}\n")
    else:
        file.write(f"Customer ID: {customer_id} | Name: {name} | Passport: {passport} | License: {license} | Address: {address} | Phone: {phone} | Registration Date: {registration_date}\n")

print(f"Customer registered successfully! Customer ID: {customer_id}")
print("Registration Date:", registration_date)
```

The `datetime.datetime.now()` retrieves the current date and time, and `.strftime("%d %B %Y")` formats this date and time as a string in the format "day month year". For example, if today is May 18, 2024, the resulting string will be "18 May 2024". After writing to the file, it prints a confirmation message to the user with the current `registration_date`.

Input Validation and User Feedback:

```
while True:
    id_type = input("Enter ID type (a for NRIC, b for Passport): ").strip().lower()
    if id_type in ('a', 'b'):
        if id_type == "a":
            while True:
                nric = input("Enter NRIC (should contain only numbers and hyphens): ").strip()
                if all(c.isdigit() or c == '-' for c in nric):
                    break
                else:
                    print("Invalid NRIC format. NRIC should contain only numbers and hyphens (-). Please try again.")
            break
        elif id_type == "b":
            passport = input("Enter passport number: ").strip()
            break
        else:
            print("Invalid choice. Please enter 'a' for NRIC or 'b' for Passport.")
    else:
        print("Invalid choice. Please enter 'a' for NRIC or 'b' for Passport.")

while True:
    phone = input("Enter customer phone number (should contain only numbers and hyphens): ").strip()
    if all(c.isdigit() or c == '-' for c in phone):
        break
    else:
        print("Invalid phone number format. Phone number should contain only numbers and hyphens (-). Please try again.")
```

This code snippets demonstrate data validation for customer ID types, NRIC (National Registration Identity Card), passport numbers, and phone numbers to ensure data integrity and provide immediate feedback to users. Initially, the system prompts the user to enter an ID type ('a' for NRIC or 'b' for Passport). If "a" is selected, a nested loop ensures the NRIC contains only numbers and hyphens by using a generator expression within the 'all' function. If the input is valid, the loop breaks; otherwise, an error message prompts the user to try again. For "b", the system accepts any input as a valid passport number. If an invalid ID type is entered, the user is prompted again. Separately, the system also validates phone numbers to ensure they contain only numbers and hyphens, using a similar validation loop. These steps ensure that user inputs conform to specified formats, enhancing the system's reliability, and preventing invalid data from being processed or stored.

Update Selective Profile Details in One Input:

```

available_fields = {"a": "name", "b": "phone_number", "c": "address", "d": "email", "e": "job_title"}
allowed_choices = ["a", "b", "c", "d", "e"]
# Prompt user for fields to update
fields_to_update = input("Enter fields to update (a: name, b: phone number, c: address, d: email, e: job title), separated by commas: ")
fields = fields_to_update.strip().lower().split(",")

update_successful = False # Flag to track if any update was successful

for field in fields: # Loop through each field to update
    field = field.strip()
    if field in allowed_choices: # Check if the field is allowed
        if available_fields[field] == "name": # Name
            new_name = input("Enter new name: ")
            current_user.name = new_name
            update_successful = True
        elif available_fields[field] == "phone_number": # Phone Number
            new_phone_number = input("Enter new phone number: ")
            if all(char.isdigit() or char == '-' for char in new_phone_number):
                current_user.phone_number = new_phone_number
                update_successful = True
            else:
                print("Invalid phone number format. Please use only numbers and hyphens.")
        elif available_fields[field] == "address": # Address
            new_address = input("Enter new address: ")
            current_user.address = new_address
            update_successful = True
        elif available_fields[field] == "email": # Email
            new_email = input("Enter new email: ")
            current_user.email = new_email
            update_successful = True
        elif available_fields[field] == "job_title": # Job Title
            new_job_title = input("Enter new job title: ")
            current_user.job_title = new_job_title
            update_successful = True
    else:
        print(f"Invalid field: {field}")

```

The `update_profile` function provides precise control over profile updates. It first checks for an existing profile and loads it. Users can then particularly choose specific fields for individual updates. An improvement could be to allow updating all details at once. This would streamline the process by letting users enter new information for all fields in one step, with a confirmation before saving.

6.0 Sample Input/Output with Explanation

Simple Input and Output:

```
Username: Ian
Password: 1234
Login successful. Welcome Customer Service Staff!
Enter your name: ian
Welcome ian!
```

The user enters "Ian" as the username and "1234" as the password. The function checks the customer_staff.txt file and finds a match for the username and password. The function returns a success message with the user's role ("Customer Service Staff") and the message is printed to the console.

After that, the user enters "ian" as their name. The user_name function creates a new UserProfile object with the name "ian" and assigns it to the current_user variable. The user is then welcomed with their name.

Arithmetic Operation:

```
MENU
1. REGISTER NEW STAFF
2. UPDATE STAFF DETAILS
3. DELETE STAFF RECORD
4. SET RENTING RATES
5. UPDATE RENTING RATES
6. VIEW MONTHLY REVENUE REPORT
7. UPDATE OWN PROFILE
Select one from (1/2/3/4/5/6/7): 6
Enter the year: 2023
Enter the month (1-12): 1
Monthly Revenue Report for 1/2023:
Total Revenue: RM1142.00
```

The user enters "2023" as the year and "1" as the month. The view_monthly_revenue function reads the car_rental.txt file and extracts the rental transactions that fall within the specified month. After that, it calculates the total revenue for January 2023 and prints the monthly revenue report with the total revenue of RM1142.00.

Conditional Statements:

If option 'a'

```
Enter customer name: John
Enter ID type (a for NRIC, b for Passport): a
Enter NRIC (should contain only numbers and hyphens): 42402-05-4053
Enter customer phone number (should contain only numbers and hyphens): 019-5349502
Enter car driving license number: A42053
Enter customer address: 10, Jalan Utama, Taman Desa
Customer registered successfully! Customer ID: C000002
Registration Date: 18 May 2024
```

The user enters "John" as the customer's name, selects option "a" for NRIC, enters "42402-05-4053" as the NRIC number, "019-5349502" as the phone number, "A42053" as the car driving license number, and "10, Jalan Utama, Taman Desa" as the address. The function generates a unique customer ID "C000002" and records the registration date as "18 May 2024". The function writes the customer information to the customers.txt file and prints a success message with the customer ID and registration date.

If option 'b'

```
Enter customer name: Bob
Enter ID type (a for NRIC, b for Passport): b
Enter passport number: AH05363
Enter customer phone number (should contain only numbers and hyphens): 013-4205224
Enter car driving license number: G042942
Enter customer address: 23, Jalan Selasih, Taman Naga
Customer registered successfully! Customer ID: C000003
Registration Date: 18 May 2024
```

The user enters "Bob" as the customer's name, selects option "b" for passport, enters "AH05363" as the passport number, "013-4205224" as the phone number, "G042942" as the car driving license number, and "23, Jalan Selasih, Taman Naga" as the address. The function generates a unique customer ID "C000003" and records the registration date as "18 May 2024". The function writes the customer information to the customers.txt file and prints a success message with the customer ID and registration date.

Loop with Input and Output:

```
Username: a
Password: a
Error: Invalid username or password. You have 2 attempt(s) remaining.
Username: b
Password: b
Error: Invalid username or password. You have 1 attempt(s) remaining.
Username: c
Password: c
Maximum login attempts reached. Please try again later.
```

The user enters "a" as the username and "a" as the password. The function checks the records in the manager.txt, customer_staff.txt, and car_service_staff.txt files and finds no match for the username and password. The function increments the attempts counter and prompts the user to enter their username and password again.

Then, the user enters "b" as the username and "b" as the password. The function checks the records in the manager.txt, customer_staff.txt, and car_service_staff.txt files again and finds no match for the username and password. The function increments the attempts counter and prompts the user to enter their username and password again.

After that, the user enters "c" as the username and "c" as the password. The function checks the records in the manager.txt, customer_staff.txt, and car_service_staff.txt files another time and finds no match for the username and password. The function increments the attempts counter and prompts the user to enter their username and password again. The maximum number of attempts (3) is reached, and the function returns an error message. The error message is printed to the console.

Date Input and Formatting:

```
Enter your choice (1-10): 2
Enter customer ID: C000001
Enter car registration number: BNL2778
Enter rental date (DD Month YYYY): 23 10 2023
Enter return date (DD Month YYYY): 24 10 2023
Invalid date format. Please use DD Month YYYY (e.g., 03 February 2024).
```

When booking a car, users provide rental and return dates in the format "DD Month YYYY" (e.g., "23 October 2023" and "24 October 2023"). These date strings are converted into `datetime` objects using `datetime.strptime` with the format string `"%d %B %Y"`. If the input format is incorrect, the program catches the error and notifies the user with a message: "Invalid date format. Please use DD Month YYYY (e.g., 03 February 2024)." Once parsed correctly, the dates are formatted back into strings using `strftime` for consistent and human-readable display, such as "23 October 2023".

Function Definition and Calling:

```
def check_login(file_name):
    try:
        with open(file_name, 'r') as file:
            for line_num, line in enumerate(file, start=1):
                if f"username: {username}" in line:
                    data = line.strip().split(', ')
                    current_user = {}
                    for info in data:
                        try:
                            key, value = info.split(': ')
                            current_user[key.strip()] = value.strip()
                        except ValueError:
                            print(f"Skipping line {line_num} with invalid data format in file '{file_name}'")
                            continue
                    if current_user.get('password') == password:
                        return f"Login successful. Welcome {current_user.get('role')}!"
    except FileNotFoundError:
        print(f"Error: The file '{file_name}' was not found.")
    except ValueError as ve:
        print(f"Error processing file '{file_name}': {ve}")
    return None

# Check for login in different roles
result = check_login('manager.txt') or check_login('customer_staff.txt') or check_login('car_service_staff.txt')

if result:
    return result
```

```
Username: Ian
Password: 1234
Login successful. Welcome Customer Service Staff!
```

The function `check\_login` is designed to authenticate user login credentials stored in different text file. It opens each file and iterates through each line to find the provided username. If found, it verifies the password and returns a welcome message with the user's role upon successful login. Otherwise, it returns `None`. Error handling is included for file not found or data formatting issues. When called with the username "Ian" and password "1234", the function successfully authenticates Ian as a Customer Service Staff member, demonstrating its effectiveness in user authentication.

7.0 Conclusion

In conclusion, the development of the Python program for the car rental system offers significant enhancements to the business processes of the company in Langkawi Island. Through the implementation of user authentication, data management, and specific functionalities tailored to each user role, the system ensures efficient operation and improved customer service. Managers can effectively oversee staff activities, set rental rates, and generate revenue reports, while customer service staff can efficiently handle customer registrations, updates, and rental transactions. Additionally, car service staff can manage car registrations, maintenance, and availability with ease. Overall, the implementation of this system promises to streamline operations, increase productivity, and enhance the overall customer experience for the car rental company in Langkawi Island.

8.0 References

GeeksforGeeks. (2024). *<https://www.datacamp.com/tutorial/converting-strings-datetime-objects>*.

Retrieved from <https://www.geeksforgeeks.org/understanding-code-reuse-modularity-python-3/>

HackerEarth. (2024). *Classes and Objects I*. Retrieved from

<https://www.hackerearth.com/practice/python/object-oriented-programming/classes-and-objects->

[i/tutorial/#:~:text=A%20class%20is%20a%20code,the%20instance%20of%20the%20class.](https://www.hackerearth.com/practice/python/object-oriented-programming/classes-and-objects-i/tutorial/#:~:text=A%20class%20is%20a%20code,the%20instance%20of%20the%20class.)

Thevapalan, A. (2023). *Python String to DateTime: How to Convert Strings to DateTime Objects*

in Python. Retrieved from <https://www.datacamp.com/tutorial/converting-strings-datetime-objects>